



A Cost-Effective Near-Storage Processing Solution for Offline Inference of Long-Context LLMs

Hongsun Jang

Seoul National University
Seoul, South Korea
hongsun.jang@snu.ac.kr

Jaeyong Song

Seoul National University
Seoul, South Korea
jaeyong.song@snu.ac.kr

Changmin Shin

Seoul National University
Seoul, South Korea
scm8432@snu.ac.kr

Si Ung Noh

Seoul National University
Seoul, South Korea
siung98@snu.ac.kr

Jaewon Jung

Seoul National University
Seoul, South Korea
jungjaewon@snu.ac.kr

Jisung Park

POSTECH
Pohang, South Korea
jisung.park@postech.ac.kr

Jinho Lee

Seoul National University
Seoul, South Korea
leejinho@snu.ac.kr

Abstract

The computational and memory demands of large language models for generative inference present significant challenges for practical deployment. One promising solution targeting offline inference is *offloading-based batched inference*, which extends the GPU's memory hierarchy with host memory and storage. However, it often suffers from substantial I/O overhead, primarily due to the large KV cache sizes that scale with batch size and context window length.

In this paper, we introduce *HILOS*, a framework that boosts offline inference throughput using near-storage processing. The core of HILOS is *attention near storage*, which offloads memory-intensive attention operations to near-storage accelerators, reducing traffic across the system interconnect. Building on attention near storage, HILOS incorporates three additional optimizations. First, *cooperative X-cache* minimizes KV cache I/O by exploiting available host resources after offloading. Second, *delayed KV cache writeback* hides storage write latency and mitigates storage write amplification. Finally, a *memory-efficient attention accelerator* sustains high throughput for long sequences within the resource constraints of NSP devices. We implemented and evaluated HILOS on a real system equipped with 16 SmartSSDs. Compared to state-of-the-art offloading-based inference frameworks, HILOS achieves up to 7.86× throughput while reducing energy consumption by up to 85%. The source code for HILOS is available at <https://github.com/hongsunjang/HILOS>.

CCS Concepts: • Computer systems organization → Heterogeneous (hybrid) systems; • Computing methodologies → Neural networks.

Keywords: Near-Storage Processing, Transformer-Based Generative Model, FPGA, Batch Inference

ACM Reference Format:

Hongsun Jang, Jaeyong Song, Changmin Shin, Si Ung Noh, Jaewon Jung, Jisung Park, and Jinho Lee. 2026. A Cost-Effective Near-Storage Processing Solution for Offline Inference of Long-Context LLMs. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '26)*, March 22–26, 2026, Pittsburgh, PA, USA. ACM, New York, NY, USA, 20 pages. <https://doi.org/10.1145/3779212.3790119>

1 Introduction

Transformer-based large language models (LLMs) [2, 19, 113, 142, 145, 162] are at the forefront of modern artificial intelligence and are expected to greatly expand human productivity [25, 42, 69]. Consequently, the efficient support of generative inference for these models [11, 18, 55, 71, 127] has become a critical challenge in modern computing systems.

While current LLM systems often focus on online inference [71, 104, 157, 167], offline inference is also critical for various scenarios [11, 100, 127]. In contrast to online inference that prioritizes low latency (i.e., low TTFT and tight TPOT constraint), offline inference generally allows longer sequences [22, 94], tolerating higher latency. It is especially useful for applications such as benchmarking [46, 47, 81, 166] and information extraction [22, 44, 96, 101, 108].

A significant challenge in offline inference is the massive memory footprint it requires. The model parameters and the key-value (KV) cache [107] for large models often exceed the memory capacity of a single GPU, especially with the long sequences common in offline tasks. In such scenarios, equipping costly servers with multiple GPUs to meet increasing memory requirements is often cost-prohibitive. Furthermore, this approach is less economical because the decoding phase of LLM inference is fundamentally memory-bound, leaving expensive GPU compute units severely underutilized [50, 102, 104, 167].

A popular direction is the *offloading-based batched inference* approach [11, 77, 127]. It generally offloads model



This work is licensed under a Creative Commons Attribution 4.0 International License.

ASPLOS '26, Pittsburgh, PA, USA

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2359-9/2026/03

<https://doi.org/10.1145/3779212.3790119>

weights and KV caches to host memory, spilling over to secondary storage if the host memory capacity is exceeded. By batching multiple tokens, it enables repeated reuse of model weights for increased throughput. However, an unsurprising consequence is the I/O overhead resulting from KV cache transfers, which is exacerbated by larger batch sizes and longer context lengths [71, 77, 127, 163]. Our study (§ 3.1) shows that KV cache I/O in the offloading-based batched inference accounts for over 60% of inference time.

Given these limitations, we advocate the use of near-storage processing (NSP) as an attractive opportunity to address this bottleneck. NSP offers several key advantages for KV cache storage: (1) abundant storage capacity for terabyte-scale KV caches, (2) high aggregate bandwidth that scales with the number of devices, (3) reduced host I/O overhead.

In this paper, we propose *HILOS*, an NSP-based framework for high-throughput offline LLM inference on a real system. The core of HILOS is *attention near storage*, which processes KV cache-related operations within a custom accelerator in each NSP device. With attention near storage, only the computed results are sent back to the host, significantly reducing the total I/O overhead. While offloading of attention operations has been explored on other platforms in previous work [50, 100, 102, 159], HILOS is the first to implement this on off-the-shelf NSP products [9, 48], addressing several practical challenges as follows.

First, offloading KV cache operations to NSP devices shifts the performance bottleneck, causing the internal storage I/O to dominate end-to-end latency. Based on the observation that this offloading leaves host resources (i.e., the GPU) underutilized, we introduce *cooperative X-cache*, a technique that stores pre-projection activations (X) and utilizes remaining host resources to further reduce the internal traffic.

Second, writing the new KV entries back to storage incurs significant latency due to small, inefficient writes. We propose *delayed KV cache writeback*, which buffers new KV entries in host memory and provides partially computed results to the accelerator. This not only puts the overhead off the critical path but also allows for efficient updates.

Third, the storage-side resources are often severely constrained, which makes it challenging to provide enough computational throughput for long-sequence attentions. To address this challenge, we design a novel architecture that efficiently utilizes off-chip memory for NSP environments.

We evaluate HILOS on an end-to-end real-system prototype built using commercial SmartSSDs [9]. Our prototype is a ready-to-use system fully integrated into a PyTorch offloading-based LLM inference framework [127]. We demonstrate that HILOS cost-effectively supports 175 billion-parameter LLMs with a 128K context length on a single A100 GPU, without requiring any custom modifications to the existing hardware or model architecture.

Our contributions can be summarized as follows:

- We introduce HILOS, the first NSP-offloading system with an off-the-shelf product that can efficiently support long-context (>100K) LLMs.
- We propose three key designs to address the limitations of the naive NSP-offloading system: (1) cooperative X-cache, which allows for cooperative utilization of host resources alongside NSP devices. (2) delayed KV cache writeback, which addresses the inefficient host-storage data transfer for new KV entries, and (3) an efficient NSP accelerator, which enables high scalability for long-context LLMs.
- We implement HILOS on a real system using commodity NSP devices (SmartSSDs [9]) and integrate it into a PyTorch-based LLM inference framework. Our evaluation demonstrates up to a 7.86× throughput improvement over conventional SSD-based solutions.

2 Background

2.1 KV Cache in Transformer-based LLM Inference

LLM generative inference generates an output sequence of length n from an input context X of length s . The process operates in two phases: *prefill* and *decoding*. After the prefill phase processes the entire input context to compute an initial *key-value (KV) cache* [107] and the first token, the decoding phase generates the remaining $n - 1$ tokens auto-regressively. At each decoding step, the KV cache is reused to attend to all previous tokens to avoid redundant computation [71, 77, 107, 127]. In each transformer layer, the input X is projected using learnable weight matrices W_Q , W_K , and W_V to generate the Query (Q), Key (K), and Value (V) matrices. The attention operation is then computed as shown in the equations below, where d denotes the hidden dimension size per head:

$$Q = X \cdot W_Q; K = X \cdot W_K; V = X \cdot W_V \quad (1)$$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V \quad (2)$$

2.2 Offloading-based Batched Inference

The substantial memory required for LLM inference often exceeds the memory capacity of multiple GPUs. For offline scenarios, offloading-based batched inference [11, 77, 100, 127] provides a low-cost alternative by extending GPU memory with host resources like CPU memory and SSDs.

Prefill Procedure. In the prefill stage (Figure 1(a)), ① the GPU loads the attention layer weights. ② the inputs are multiplied by these weights to generate the query, key, and value matrices (QKV), optionally followed by a positional embedding [135]. ③ These matrices are used to calculate attention scores for the input prompt sequence via multi-head self-attention computation (ATN). ④ Next, the MLP weights are loaded from host memory. ⑤ The final output activations of the transformer block are generated (MLP). ⑥ The generated KV cache is written back to the host memory or to SSD. Steps ①–⑥ are repeated for each transformer block.

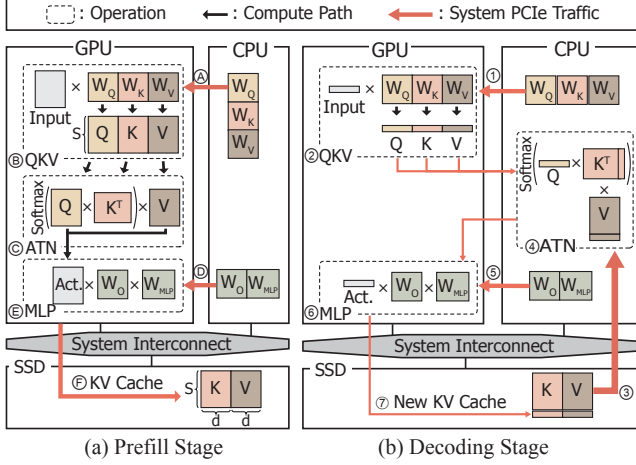


Figure 1. Procedure of the baseline offloading-based batched inference [11, 127] for the (a) prefill and (b) decoding stages.

Decoding Procedure. In the decoding stage (Figure 1(b)), ① the GPU retrieves the attention layer weights and the output of the previous layer. ② The GPU performs the QKV projection. ③ The stored KV cache for each layer is loaded. ④ The self-attention (ATN) is performed using the loaded KV cache. ⑤ The weights (W_O, W_{MLP}) for the MLP computation are loaded onto the GPU. ⑥ The GPU processes the MLP computation. ⑦ The KV cache for the newly generated token is written back to its original location for future iterations. This sequence of steps (①–⑦) is repeated for each transformer block until sequence completion.

2.3 Near-Storage Processing Devices

Computational storage devices (CSDs) [1, 63, 78, 89, 100, 124, 125, 161] have been extensively studied over the past decades. By co-locating compute units with storage, CSDs reduce data transfer latency and alleviate I/O bandwidth bottlenecks. Building on this line of research, commercial implementations [9, 131, 149] have converged on a near-storage processing (NSP) architecture, which typically features a lightweight accelerator (e.g., an FPGA) connected to the storage media via a dedicated internal PCIe interconnect.

A state-of-the-art example is Samsung’s SmartSSD [9], which features on-board DRAM that serves as a cache for the NAND flash and is accessible by both the host CPU and the device’s integrated FPGA. The host can perform standard I/O to the SSD or offload computational kernels (e.g., device binary file) to the FPGA via OpenCL APIs [154]. It leverages an internal P2P PCIe path for direct data movement between the NAND flash and on-board DRAM [17, 160].

3 Motivation

This section identifies KV cache I/O as the primary bottleneck in offloading-based batched inference and motivates our NSP-based solution. Using the half-precision OPT-175B [162],

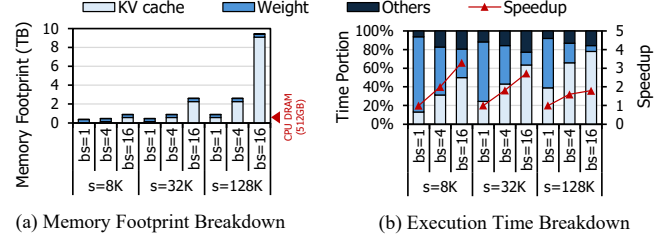


Figure 2. Motivational experiments with a 175B model.

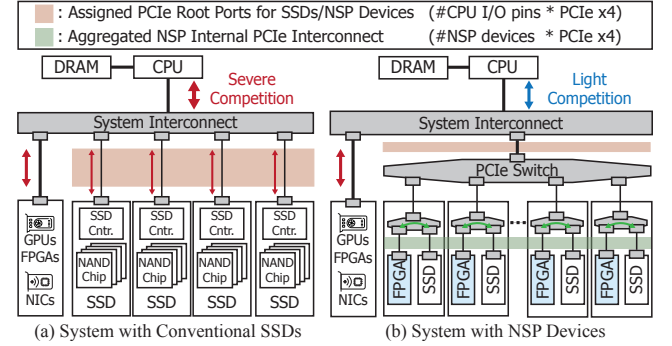


Figure 3. An example of a PCIe topology with (a) conventional SSDs and (b) near-storage processing devices.

we analyze performance across varying context lengths (s) and batch sizes (bs); see § 6.1 for setup details.

3.1 Performance Bottleneck Analysis

Many existing systems [11, 77, 127] employ large-batch offline inference to mitigate the frequent model weight transfer overhead. This approach, however, suffers from the KV cache growing proportionally to both batch size and context length.

We demonstrate this bottleneck on a state-of-the-art offline inference system [127] on an A100 server (512GB host memory, four PCIe 4.0 SSDs). The results in Figure 2(a) show that the KV cache dominates the system’s memory footprint, reaching a terabyte scale that exceeds the typical host memory capacity. More critically, Figure 2(b) reveals that for long-context inference, the system becomes overwhelmingly bottlenecked by data movement. Transferring the KV cache consumes over 60% of the total execution time. Consequently, this diminishes the advantage of the conventional batching strategy, as there is less room for batching to improve from a reduced portion of weight transfer.

3.2 Advantages of Near-Storage Processing

The study above reveals that the KV cache transfer overhead is a fundamental bottleneck for offloading-based batched inference. This motivates exploration of an NSP solution to address this challenge through several key advantages.

First, the large capacity of NSP devices accommodates the terabyte-scale KV caches. Second, NSP mitigates the host

PCIe bandwidth bottleneck inherent in conventional multi-SSD systems (Figure 3(a)). By processing data near storage, NSP minimizes host-device data transfers (Figure 3(b)). Third, NSP alleviates contention for host resources already heavily utilized by weight loading and self-attention computation.

Although NSP appears promising, a naive deployment does not guarantee performance improvements. Realizing its full potential requires addressing several key challenges: (1) determining which computations to offload (§ 4.1), (2) efficiently scheduling tasks between the host and NSP devices (§ 4.2), (3) effectively hiding storage write latency (§ 4.3), and (4) designing a lightweight yet high-throughput storage-side accelerator (§ 4.4). We address these challenges with a complete system built from off-the-shelf components [9, 48].

4 HILOS Design

4.1 Attention Near Storage

As discussed in § 3, the offloading-based batched inference suffers from substantial KV cache communication overhead during decoding. With conventional storage devices, this communication must traverse the shared system interconnect. HILOS addresses this issue with *attention near storage* (ANS), which offloads attention to a custom accelerator near storage. It confines the high-volume KV cache traffic to the device's internal path, allowing only the attention inputs and outputs to traverse the shared system interconnect.

As illustrated in Figure 4(a), ANS targets the decoding stage of the inference. ① The GPU loads the required LLM weights from the host memory and ② performs the QKV projection. ③ The resulting query, key, and value vectors are written to the SSD. ④ The near-storage accelerators load the full KV cache from local storage to their DRAM via a private path, ⑤ perform the attention computation, and transfer the final attention output back to the host. ⑥ The GPU then loads MLP weights from the host memory and ⑦ executes the subsequent MLP layers.

This strategy yields a significant reduction in data movement. With a context length s and hidden dimension h using half precision, the baseline's interconnect read traffic for the KV cache is $4 \cdot s \cdot h$ bytes per decoding step. Attention near storage eliminates this traffic, replacing it with a transfer of only the $2 \cdot h$ -byte attention output vector from the device to the host. This benefit comes at the cost of a minor increase in write traffic, from $4 \cdot h$ bytes in the baseline (new K and V) to $6 \cdot h$ bytes in attention near storage (new Q, K, and V).

The overall traffic reduction is substantial because the dominant read traffic scales with the context length s . Comparing the baseline traffic T_{BASE} , and the traffic with attention near storage T_{ANS} , the benefit grows linearly with s :

$$\frac{T_{BASE}}{T_{ANS}} = \frac{4sh + 4h}{2h + 6h} = \frac{s + 1}{2} > 1 \quad (\because s > 1). \quad (3)$$

To scale this approach to multiple NSP devices, we parallelize attention along both the batch and attention head

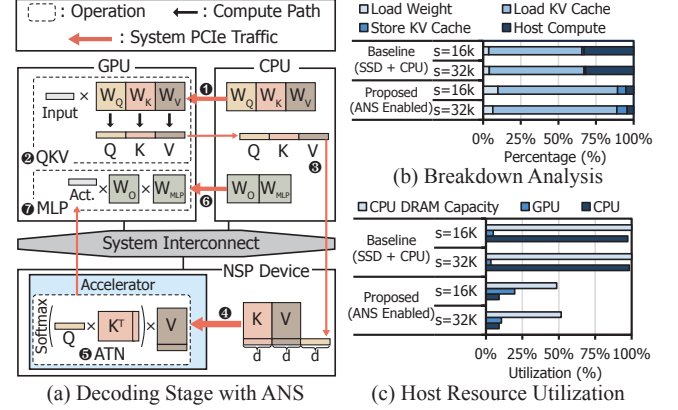


Figure 4. (a) Proposed decoding workflow with an NSP device. (b) Latency breakdown of the decoding stage. (c) Host resource (CPU, GPU, DRAM) utilization comparison.

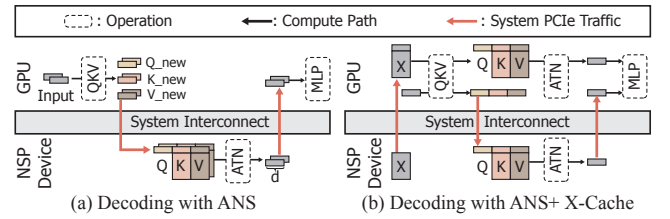


Figure 5. Decoding procedure with (a) only ANS and (b) both ANS and cooperative X-cache.

dimensions. With batched inference, the product of the batch size and the number of attention heads provides sufficient dimensions to distribute the workload across the available NSP devices. For this, the prefill stage distributes the KV cache in the same parallelization across the NSP devices.

Our approach follows the principle of near-storage processing [1, 9, 63, 78, 161] and effectively reduces interconnect traffic. However, in practice, the offloading of attention operations to NSP shifts the performance bottleneck, now causing the internal storage I/O to dominate end-to-end latency, as shown in Figure 4(b). This poses a key challenge for practical NSP systems: reducing internal storage I/O. Fortunately, the offloading leaves the host severely underutilized (<20%, Figure 4(c)), with the host remaining active only during the computationally inexpensive QKV projection and MLP stages. Based on this observation, the following section proposes a strategy to ensure effective cooperative execution between the host and the near-storage device, addressing the aforementioned challenge.

4.2 Cooperative X-Cache

To reduce the internal storage I/O by exploiting the host underutilization of the baseline procedure (Figure 5(a)), we propose a cooperative execution schedule that partitions

the attention workload between the host and the NSP devices. Recalling from Equation (1) that the key and value caches are projections of the input activation X , we cache the pre-projection input activation X for the historical context instead of the derived K and V tensors. We term this technique *cooperative X-cache*.

Figure 5(b) details the procedure. We define α as the fraction of batches managed by X-cache. The remaining $1 - \alpha$ portion utilizes the standard KV cache stored in the SSD. This parameter α partitions the batch (b) and head (h) dimensions rather than the sequence dimension (s). During the prefill phase, the system persists the input activations X corresponding to the α portion. In the decoding phase, the GPU loads these X-cache entries directly via GPU-Direct Storage (GDS) [98] to minimize system traffic. The GPU subsequently regenerates the required K and V tensors by re-executing the projection operations (Equation (1)). Simultaneously, the near-storage accelerator (§ 4.1) performs multi-head attention on the $1 - \alpha$ portion of the KV cache.

Since X-cache ($b \times h \times s \times d$) requires only half the storage capacity of the combined key and value tensors ($2 \times b \times h \times s \times d$), it effectively halves both the system-interconnect and flash-read traffic. This presents a favorable trade-off, as it utilizes the GPU's high processing power that would otherwise remain idle while attention operations are offloaded. Since this recomputation is executed concurrently with the attention operation on the NSP devices, its latency is effectively hidden. In addition, this positively affects the SSD endurance from less writes to the SSDs (§ 6.6).

I/O Analysis. To quantify the impact of cooperative X-cache, we derive a first-order cost model. Assuming 16-bit precision (2 bytes per element), the time to transfer the X-cache portion (α) from the NSP device to the GPU over the host interconnect with bandwidth B_{PCI} is

$$T_{PCI} = \alpha \cdot s \cdot h \cdot 2 / B_{PCI}.$$

Let C_{GPU} denote the GPU's compute capability in FLOPS. To regenerate K and V , the input $X \in \mathbb{R}^{s \times h}$ is multiplied by the weight matrices $W_K, W_V \in \mathbb{R}^{h \times h}$. The time required for this additional recomputation per transformer block is

$$T_{GPU} = \alpha \cdot s \cdot h^2 \cdot 2 / C_{GPU}.$$

The storage read time is governed by the SSD's bandwidth, B_{SSD} , as a sum of reading the X-cache data (α fraction) and the KV-cache data ($1 - \alpha$ fraction):

$$T_{SSD} = (\alpha \cdot (s \cdot h \cdot 2) + (1 - \alpha) \cdot (2 \cdot s \cdot h \cdot 2)) / B_{SSD}.$$

Assuming that the regeneration computation and data transfers are well-pipelined and can be overlapped,

$$T_{\text{effective}} = \max(T_{GPU}, T_{SSD}, T_{PCI}).$$

Because T_{GPU} is often small, an α satisfying $T_{SSD} = T_{PCI}$ typically yields the best latency. Let S_X denote the size of

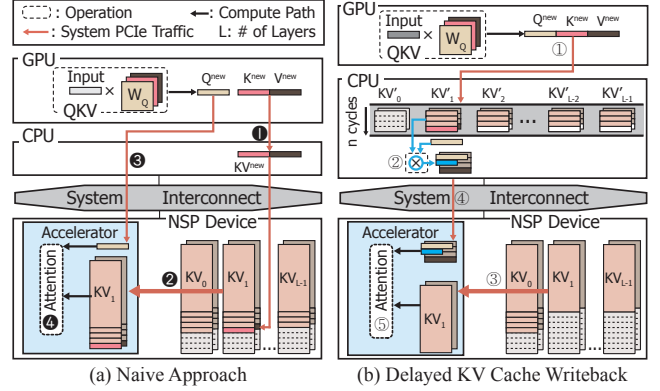


Figure 6. KV-cache write-back procedure with (a) naive CSD offloading and (b) delayed KV cache writeback.

X , which implies the KV-cache size is $S_{KV} = 2 \cdot S_X$. Setting $T_{PCI} = T_{SSD}$ yields

$$\frac{\alpha S_X}{B_{PCI}} = \frac{\alpha \cdot S_X + (1 - \alpha) \cdot S_{KV}}{B_{SSD}} = \frac{\alpha \cdot S_X + 2 \cdot (1 - \alpha) \cdot S_X}{B_{SSD}}.$$

Canceling S_X and solving for α yields

$$\alpha = \frac{2B_{PCI}}{B_{SSD} + B_{PCI}}.$$

This implies that the ratio between B_{SSD} and B_{PCI} determines the optimal α . We model B_{SSD} to scale linearly with the number of NSP devices, while the effective bandwidth B_{PCI} is profiled over several layers. We use this formulation to automatically select an α closest to a power of two.

4.3 Delayed KV Cache Writeback

While attention near storage and cooperative X-cache improve overall throughput, handling write-backs during decoding phase remains challenging. During the prefill phase, KV caches follow a *row-wise layout* ($b \times h \times s \times d$). The X-cache also employs the same mechanism, sharing an identical size and layout with the K-cache. This layout ensures high SSD bandwidth because the minimum access granularity ($s \times d$) typically exceeds 4KiB. Conversely, each decoding iteration generates small KV vectors ($1 \times d$) that must be appended to the existing cache ($s \times d$) on storage. For these small vectors to be truly available for subsequent attention operations, they must be committed to storage. This dependency places storage write latency on the critical execution path and incurs an inefficient sub-page granularity writes.

In the naive approach shown in Figure 6(a), after the GPU generates new KV entries, ① each entry is individually written to the SSD via direct I/O. ② Subsequently, the processing unit reads the entire updated KV cache from storage to its local memory. ③ Once the new query vector is transferred to the processing unit, ④ the attention operation is performed. In this method, SSD writes lie on the critical execution path.

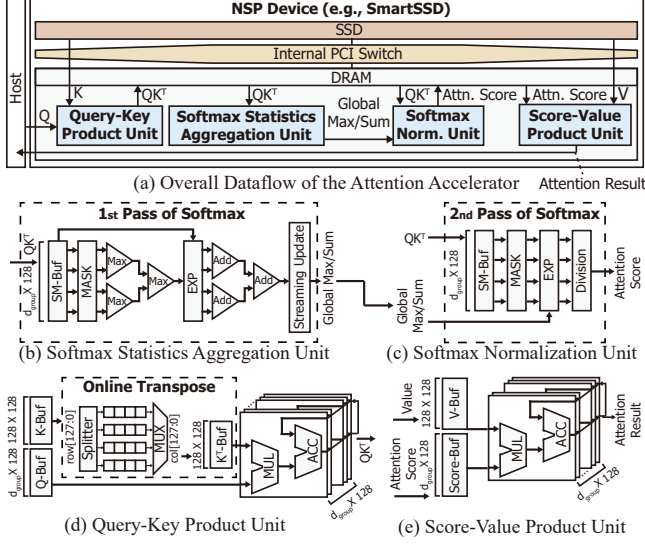


Figure 7. Dataflow and units of the attention accelerator.

Moreover, each KV entry (256 bytes) is far smaller than the SSD page size (4KiB), leading to poor write performance.

To address this, we propose *delayed KV cache writeback*, shown in Figure 6(b). Instead of writing every newly generated KV entry directly to storage, we keep dedicated buffers for KV data in host memory and allow the buffer to feed directly into the accelerator’s off-chip memory. From this, we obtain the benefits of (1) the write latency stays off the critical path, and (2) data are written in page-sized chunks.

A remaining issue from the delayed KV cache writeback is that the same KV vectors are sent redundantly until they are spilled. To mitigate this overhead, the host CPU precomputes the partial dot products (QK^T) between the current query and the buffered key vectors. Only the resulting scalar values (marked blue) are sent to the processing unit.

The optimized workflow proceeds as follows: ① New KV entries are staged in host memory buffers. ② The CPU precomputes the partial QK^T scores from these buffered entries. ③ The already stored K and V tensors are read from the SSD into the accelerator. ④ The query, the precomputed QK^T scalars, and the new V entries are transferred from host memory to the accelerator. ⑤ The accelerator uses these components to complete the final attention computation. The buffered values are later spilled to the storage after a predetermined *spill interval*. Given that the SSD write granularity is 4KiB and each KV entry is typically 256 bytes per head [36, 37, 60, 83, 142, 143, 162], performance optimization is typically achieved with the spill interval set to 16.

4.4 Attention Accelerator Architecture

While there are several existing transformer accelerators [23, 50, 52, 102, 104, 159], HILOS designs a new architecture to address the following challenges for near-storage processing:

Algorithm 1: Efficient Softmax Implementation

```

1  $m \leftarrow -\infty, Z \leftarrow 0$ ; // Initialize running max and sum
  // First pass: compute global statistics
2 foreach block  $B$  in input sequence  $x$  do
3    $B \leftarrow \text{MASK}(B); m_B \leftarrow \max_{b \in B} b$ ;
4    $S_B \leftarrow \sum_{b \in B} \exp(b - m_B)$ ;
  // Streaming Update Unit
5   if  $m_B > m$  then
6      $Z \leftarrow Z \cdot \exp(m - m_B) + S_B$ ;
7      $m \leftarrow m_B$ ;
8   else
9      $Z \leftarrow Z + S_B \cdot \exp(m_B - m)$ ;
  // Second pass: normalize element by element
10 foreach  $x_i$  in input sequence  $x$  do
11    $x_i \leftarrow \text{MASK}(x_i); y_i \leftarrow \exp(x_i - m)/Z$ ;

```

1. *Limited available resources.* Near-storage accelerators have tight resource budgets. For example, Samsung SmartSSD employs a moderately sized FPGA, which is further limited by the SSD power budget [105]. This becomes especially problematic for softmax operations for long contexts.
2. *Layout conflict for key writes and reads.* As described in § 4.3, the new KV entries are added to the existing KV cache in a row-wise manner. However, the key matrix has to be read in columns (i.e., K^T for the attention), creating a mismatch.
3. *Support for attention variants.* Modern LLM architectures are increasingly adopting variants of attention such as group-query attention (GQA). While it can be processed with existing architectures for multi-head attention, it would cause redundant data accesses.

Figure 7(a) depicts the overall dataflow of the proposed accelerator. Existing spatial architectures [106, 144, 152] compute the entire attention operation in a single pass, requiring on-chip memory capacity that grows prohibitively large with sequence length [23, 100]. In contrast, we adopt a temporal architecture [52, 64, 79] that processes the attention computation in blocks (e.g., 128 tokens per block). As illustrated in Figure 7(a), four hardware units form a concurrent pipeline to process each block in accordance with data dependencies. This choice substantially reduces the resource requirement, especially the on-chip memory.

Two-Pass Softmax. Given an input vector $x = [x_1, \dots, x_s]$, the softmax operation is

$$\text{softmax}(x) = [o_1, \dots, o_s],$$

$$\text{where } o_i = \frac{e^{x_i - \max(x)}}{\sum_{j=1}^s e^{x_j - \max(x)}} \quad \text{for } i = 1, \dots, s. \quad (4)$$

To avoid numerical instability, this is typically done in three passes [95]: one to find the global maximum $\max(x)$, a second to compute the sum of exponentials, and a third for

element-wise normalization. For long sequences, this method creates a major bottleneck from the off-chip memory traffic.

To address this, inspired by the online softmax algorithm [30, 31, 95], we design a two-pass (instead of three) method as in Algorithm 1. The two-pass softmax is conducted via two units: softmax statistics aggregation unit (Figure 7(b)) and softmax normalization unit (Figure 7(c)).

In the first pass (Figure 7(b)), the statistics aggregation unit processes the input vector in blocks of $d_{group} \times 128$ elements. Each block is streamed through a pipelined max reduction tree to compute a local maximum (line 3). The key enabler of this two-pass algorithm is that this local maximum is used instead of the global maximum to stabilize the exponential values by feeding it into parallel exponentiation units. The results are aggregated by the adder tree to compute a partial sum (line 4). Then, depending on whether the global maximum is updated, the streaming update unit modifies the global sum, which serves as the final denominator (lines 5-9). The second pass (Figure 7(c)) performs the normalization, composed of element-wise exponential and division (lines 10-11). For the masking of the attention calculation, MASKs in both units apply padding masks and precomputed scalars from the host (§ 4.3) (lines 3, 11) for each input element.

GEMV with Online Transpose. The GEMV units perform the query-key (Figure 7(d)) and attention score-value (Figure 7(e)) products. A central challenge in the query-key computation is the memory layout transformation for the transposed Key matrix (K^T). Naive solutions would be to store in a transposed format K^T or in both. However, they would be inefficient for SSD writes or add redundant writes.

Leveraging the block-based mechanism, our design integrates an efficient in-place transposition module, as shown in Figure 7(d). By simply performing a local block transpose and matching it with the right Q block, global transpose can be avoided. Thus, the unit loads a 128×128 square block of the Key matrix into an on-chip buffer (K-Buf), performs the transposition locally, then stores the transposed matrix to another on-chip buffer (K^T -Buf). Finally, the transposed matrix is streamed to the MAC units for a blocked GEMV.

Native Support for Attention Variants. Our architecture also provides native support for attention variants such as GQA [37, 140, 143], where d_{group} query heads share a single KV cache. For this, the K/V buffers broadcast their values to the $d_{group} \times 128$ parallel MAC units to concurrently process d_{group} query vectors that share access to the same KV cache data. Without this feature, the shared KV Cache would be read redundantly for each query in a group. The partial results are accumulated into each final output buffer per query, reducing redundant off-chip memory traffic.

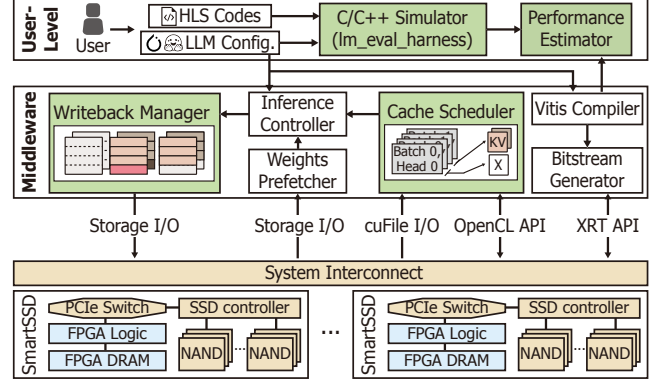


Figure 8. Overview of HILOS, a ready-to-use framework.

5 Full System Integration

We prototype HILOS on a real system using commercially available SmartSSDs and a PyTorch-based framework. Figure 8 illustrates the overview of HILOS, which comprises three main components: a user-level design flow for accelerator customization (top), a middleware layer for HILOS (middle), and the underlying hardware testbed (bottom).

5.1 User-Level Design Flow

HILOS provides an end-to-end design flow for generating FPGA bitstreams for the SmartSSDs, as depicted in the top of Figure 8. While we provide pre-compiled binaries for common use cases, users can customize the high-level synthesis [10, 90] (HLS) source code to implement specialized attention variants [83, 139]. For easy customization, we offer a simulation and verification tool integrated with the lm-evaluation-harness [40]. This allows users to validate the functional correctness of accelerator designs on standard LLM benchmarks [15, 91, 94, 119] before committing to resource-intensive synthesis. Furthermore, we provide a performance estimator for the accelerator based on the cycle counts and the clock frequency obtained from HLS. Across sequence lengths from 4K to 32K, the estimator achieves a Pearson correlation coefficient of 0.93 compared to measured hardware throughput for the three kernels (Table 3). This high correlation demonstrates that the simulator accurately captures the performance trends of the target hardware.

5.2 Middleware Layer

The middleware layer (Figure 8) orchestrates the LLM inference pipeline. During runtime, the *Inference Controller* directs core LLM operations (e.g., MLP, QKV) while the *Weights Prefetcher* stages model weights from the host to the GPU. The *Cache Scheduler* (§ 4.2) determines an optimal X-cache ratio α , partitions the KV cache between SmartSSDs and the GPU during prefill, and coordinates parallel execution during

decoding. Concurrently, the *Writeback Manager* (§ 4.3) asynchronously spills data to the SmartSSDs upon reaching a configured threshold, preventing LLM inference pipeline stalls. These C++ runtime components interface with PyTorch via pybind11 [109] and are packaged as a callable Python module using distutils [110]. An offline toolchain, featuring the *Vitis Compiler* and *Bitstream Generator*, synthesizes custom HLS designs into FPGA bitstreams for deployment.

5.3 Hardware Testbed

The HILOS testbed consists of up to 16 SmartSSDs, each equipped with our custom accelerator (§ 4.4). These devices are integrated into a host system via a general-purpose PCIe expansion chassis [48] that supports 16 PCIe 4.0 lanes. To enable peer-to-peer (P2P) communication [17, 160], we allocate buffers in the FPGA’s on-board DRAM using the Xilinx OpenCL extension CL_MEM_EXT_PTR_XILINX. This flag makes the buffer memory-mapped and accessible to the CPU, FPGA, and the SSD controller, facilitating direct data transfers. Each FPGA connects to its corresponding SSD via an internal PCIe switch, ensuring that all communication remains within the SmartSSD and avoids inter-device traffic.

5.4 FPGA Implementation Details

This section details the FPGA logic implementation and the specific design decisions for hardware bottlenecks.

Computation Design and Hardware Trade-offs We utilize the AMD Vitis HLS Math Library for hardware optimized cmath operations, including exponential functions and data type conversions. Conversely, we develop the softmax and GEMV modules from scratch to enable fine-grained architectural control within the resource constraints of a near-storage accelerator. A primary design decision involves selecting unrolling factors that maximize throughput without exceeding hardware limits. Specifically, we apply a loop unrolling factor of two to the exponential units. This configuration balances the high DSP consumption of exponential operations with the necessity for parallel execution. Furthermore, we implement maximum value and sum-of-exponential reductions using a two-level tree structure with a depth of four. This design choice minimizes pipeline latency while ensuring timing closure for the high-latency four-way reduction logic. The GEMV operations are parallelized across 128 MAC units. While the hardware resources permit higher degrees of parallelism, we utilize 128 units because this configuration successfully saturates the available DRAM bandwidth.

Memory Access Optimization To maximize hardware interface efficiency, input sequences are zero-padded to multiples of 32 to facilitate AXI burst transactions. We utilize the HLS Vector Library to implement 32 element bursts of 16 bit half precision data to align each transaction with the 512 bit AXI4 data width. This configuration leverages the maximum bitwidth supported by the hardware to facilitate high throughput memory access. We also apply cyclic partitioning

Table 1. Experimental Environments

HW	GPU	NVIDIA A100 (40GB), H100 (80GB)
	CPU	Xeon(R) Gold 6342, 24C 48T
	Memory	16×32GB DDR4-3200
	CSD	SAMSUNG SmartSSD [9], 3.84TB
	Conventional SSD	SAMSUNG PM9A3 [121], 3.84TB
SW	PCIe Expansion	H3 Falcon 4109
	OS / Python	Ubuntu 20.04 LTS / 3.12.4
	PyTorch / DeepSpeed	2.4.1 / 1.15.1
	Vitis / XRT	2023.1 / 2.12.427
	CUDA / OpenCL	12.1.105 / 2.2

Table 2. Model Configurations

Model	# layers	Hidden	Interm.	# heads	# KV heads	d_{group}	# experts
OPT-30B	48	7,168	28,672	64	64 (MHA)	1	-
OPT-66B	64	9,216	36,864	72	72 (MHA)	1	-
OPT-175B	96	12,288	49,152	96	96 (MHA)	1	-
Qwen2.5-32B	64	5,120	27,648	40	8 (GQA)	5	-
Mixtral-8x7B	32	4,096	14,336	32	8 (GQA)	4	8
GLaM-143B	32	4,096	16,384	32	32 (MHA)	1	64

with a factor of 32 to the BRAM buffers for DRAM reads to match the 512-bit granularity. These techniques collectively ensure that the DRAM bandwidth is fully utilized.

Dataflow Integration We apply the DATAFLOW pragma to the top-level kernel to enable task-level pipelining. This allows the system to overlap KV cache loading with the computation of preceding segments. To meet strict dataflow constraints, our design follows a forward-only data propagation strategy. For example, although the softmax input/output buffers and the KV cache buffers share identical dimensions, we allocate dedicated hardware resources for each module rather than employing shared buffers. This design choice eliminates memory access contention and satisfies the requirements of the DATAFLOW pragma, enabling the HLS compiler to achieve effective task-level parallelism.

Numerical Precision and Stability. The design utilizes native FP16 data types for storage while performing intermediate calculations in FP32. Specifically, accumulations and exponential operations use FP32 to preserve numerical stability. During the attention mechanism, a masking module assigns a constant value of -10^4 to padding tokens. This ensures that these tokens do not influence softmax results, thereby maintaining numerical stability during inference.

6 Evaluation

6.1 Experimental Setup

Hardware Configuration. Our experimental setup is detailed in Table 1. Each SmartSSD [9] includes a 3.84TB NVMe SSD and a Kintex UltraScale+ KU15P FPGA [153] attached with 4GB of DDR4-2400 DRAM. Our setup also includes either an A100 or H100 GPU, connected to the CPU via PCIe

Table 3. Resource Utilization and Achieved Performance

Type	LUT	FF	BRAM	URAM	DSP	Peak Perf.	Power
$d_{\text{group}} = 1$	38.76%	28.57%	51.02%	9.38%	10.06%	11.9 GFLOPS	11.25 W
$d_{\text{group}} = 4$	56.60%	39.70%	59.30%	9.38%	20.27%	46.8 GFLOPS	15.39 W
$d_{\text{group}} = 5$	67.40%	46.15%	58.49%	9.38%	27.79%	56.3 GFLOPS	16.08 W

4.0×16 lanes. The baseline systems are equipped with four 3.84TB SAMSUNG PM9A3 SSDs [121], each providing up to 6,900MB/s read and 4,100MB/s write bandwidth. Each SSD is connected to the host via a dedicated PCIe 4.0 x4 link, occupying a total of 16 PCIe lanes for storage—matching the total number of host PCIe lanes used by the SmartSSDs.

Inference System Configurations. We compare HILOS against two state-of-the-art offloading-based batched inference frameworks, FlexGen [127] and DeepSpeed [11, 51].

We evaluate three FlexGen configurations for KV cache offloading: to host DRAM (**FLEX(DRAM)**), to four PCIe 4.0 SSDs (**FLEX(SSD)**), and to the 16 NVMe SSDs in our SmartSSD platform with the FPGAs disabled (**FLEX(16 PCIe 3.0 SSDs)**). For DeepSpeed, we use its ZeRO-Inference engine [11] and extend it with Unified Virtual Memory [8] (**DS+UVM(DRAM)**). This extension was necessary to manage the high GPU memory pressure of intermediate activations for long-context inference, as this functionality is not natively supported. Our system, **HILOS (N SmartSSDs)**, is configured with N SmartSSDs (N=8 by default), an X-cache ratio derived from § 4.2, and a storage spill interval of 16. To ensure fair comparisons, all systems utilize FlashAttention [30, 31] for the prefill stage. All baseline storage configurations use a software RAID-0 using *mdadm* [97], and all baselines offload attention computation to the CPU [11, 127] during decoding.

Workload. We evaluate representative decoder-only models listed in Table 2. Our evaluation includes models with standard multi-head attention (MHA), such as OPT-30B, 66B, and 175B [162], which we refer to by their parameter counts. We also evaluate models with GQA [4], including Qwen2.5-32B [140], and Mixture-of-Experts (MoE) models like Mixtral-8×7B [60] and GLaM-143B [36], with two active experts per layer. Although some models were pre-trained with shorter context lengths, we benchmark performance up to 128K tokens to highlight our system’s long-context potential, anticipating future models with extended contexts [2, 37, 139]. We use a default batch size of 16, FP16 precision, and a generated output length of 64 tokens. Weights reside in CPU memory when capacity permits, whereas models exceeding 100B parameters are offloaded to storage.

6.2 Implementation Results

Figure 9(a) shows our hardware testbed, which integrates 16 SmartSSDs via a PCIe switch. The switch connects to the host using a PCIe x16 link and provides eight PCIe x8 ports, each driving two SmartSSDs through a U.2 adapter. Figure 9(b) presents the FPGA resource floorplan, and Table 3 details

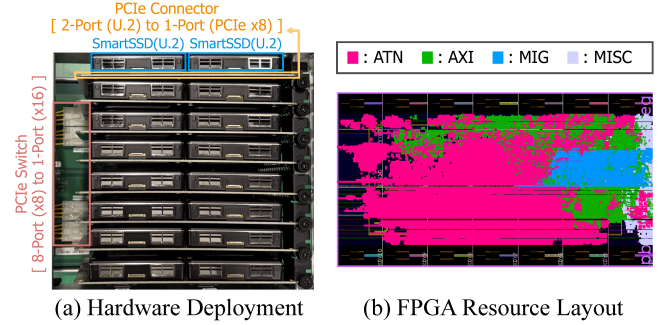


Figure 9. (a) Hardware deployment of the proposed system, featuring Samsung SmartSSDs [9] interconnected via a PCIe. (b) The corresponding FPGA resource floorplan of the user logic partition (ULP) region, for a $d_{\text{group}} = 1$ configuration.

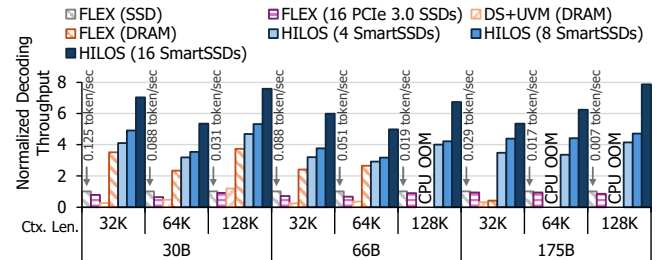


Figure 10. Performance comparison of HILOS and baselines.

resource utilization, peak performance, and total on-chip power consumption. The attention module is the primary consumer of resources. Specifically, the softmax unit utilizes a large fraction of DSP blocks for exponential computations. In contrast, GEMV units primarily utilize LUTs to manage complex memory transactions such as transposition.

The reported on-chip power, comprising static, dynamic, and PCIe transceiver power, peaks at 16.08 W per accelerator at $d_{\text{group}} = 5$. Consequently, a full 16-accelerator deployment consumes approximately 258 W, which is comparable to the power usage of a single mid-range GPU. Increasing d_{group} improves throughput by sharing the key-value cache across multiple queries, but this comes at the cost of increased resource and power consumption. The design achieves an operating frequency of 296.05 MHz. This frequency approaches the 300 MHz limit imposed by the strict power envelope of the SmartSSD platform.

6.3 Performance Comparison

Figure 10 compares the LLM decoding throughput of HILOS against four baselines across model sizes and context lengths. All results are normalized to FLEX(SSD). Since the DS+UVM(DRAM) offloads activations using UVM [8], its overhead results in a slowdown of over 4× relative to FLEX(DRAM). The FLEX(16 PCIe 3.0 SSDs) baseline, using 16 SmartSSDs

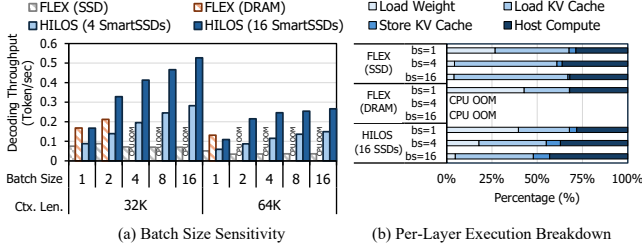
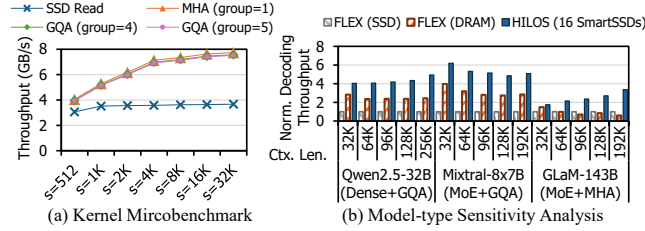


Figure 11. Sensitivity to batch size with 66B model. (a) Decoding throughput. (b) Per-layer execution breakdown.



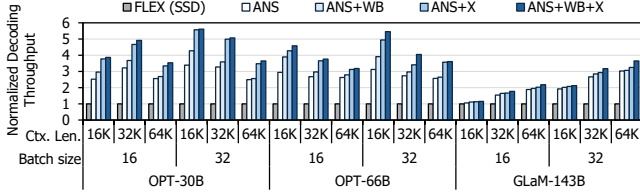


Figure 15. An ablation study of HILOS.

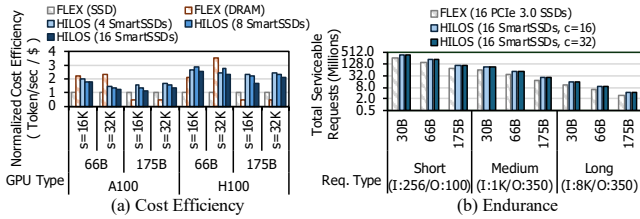


Figure 16. (a) Cost-effectiveness and (b) endurance analysis.

6.08 $\times$ over the baseline. This is because the fixed prefill latency is amortized with longer outputs. Because the delayed KV cache writeback mechanism leverages a write-back interval, c , efficient scaling for long sequences is ensured.

6.5 Ablation Study Results

Figure 15 presents an ablation study of our optimizations, with throughput normalized to the FLEX(SSD) baseline. Our ANS configuration, which excludes the write-back (delayed KV cache writeback) and X-cache optimizations, provides up to a 3.39 $\times$ speedup. Enabling the delayed KV cache writeback mechanism (ANS+WB) yields an additional speedup of up to 1.32 $\times$ over ANS by hiding storage write latency. The X-cache optimization (ANS+X) delivers a more substantial gain, achieving up to a 1.64 $\times$ speedup over ANS. Although the overall improvements for the MoE model GLaM-143B are more modest due to its lower KV-cache-to-weight ratio, the relative benefits of our techniques scale more effectively with both longer context lengths and larger batch sizes.

6.6 Cost and Efficiency Analysis

Cost-Effectiveness. Figure 16(a) presents a comprehensive analysis of the cost-effectiveness, measured in tokens per second per dollar (tokens/sec/\$), normalized to FLEX(SSD). The evaluation uses a baseline server with a \$15,000 host server and a \$7,000 A100 GPU. The HILOS configuration adds a \$10,000 PCIe expansion and sixteen SmartSSDs at \$2,400 each to this system, replacing conventional \$400 PCIe 4.0 SSDs. For the 66B model, HILOS achieves up to 2.02 $\times$ higher cost-efficiency than FLEX(SSD). While FLEX(DRAM) is 1.53 $\times$ more cost-effective when host DRAM is sufficient, its advantage diminishes for larger models. For the 175B model, where DRAM capacity limits FLEX(DRAM), HILOS improves cost-efficiency by up to 1.68 $\times$. In comparison, upgrading

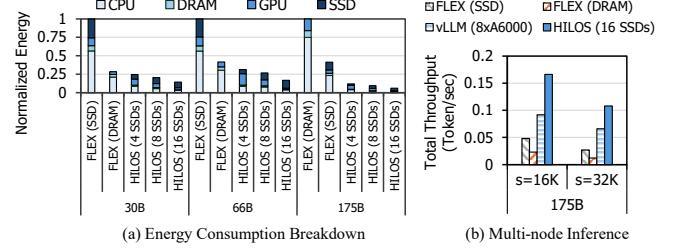


Figure 17. (a) Energy consumption breakdown analysis. (b) Comparison with multi-GPU solutions using vLLM [71].

the GPU to a \$30,000 H100 provides a 1.39 $\times$ speedup, but I/O bottlenecks reduce its cost-efficiency. HILOS delivers a comparable 1.29 $\times$ speedup while achieving 2.91 $\times$ greater cost-efficiency than the H100 configuration.

Endurance Analysis. Storage write endurance is often a cost concern, but HILOS can avoid early SSD replacement due to two reasons. First, the LLM inference workload exhibits a *write-once, read-many* access pattern for the KV cache, where endurance is mainly limited by the total write volume. Second, HILOS employs two techniques to reduce total write traffic. The X-cache mechanism, with a cache rate of $\alpha\%$, lowers storage writes by approximately $\frac{\alpha}{2}\%$, since only half of the KV cache (X) needs to be stored. Furthermore, delayed KV cache writeback defers writes for newly generated KV cache entries, reducing write amplification.

Figure 16(b) presents our endurance evaluation. We classify requests as Small (I:256/O:100), Medium (I:1K/O:350), and Long (I:8K/O:350) based on Azure workload statistics [134]. We estimate the total serviceable requests with 16 SmartSSDs, where each 3.84 TB SSD supports 7.008 PBW with a 3-month data retention [9]. Compared to the baseline, HILOS improves endurance by 1.34 $\times$ to 1.47 $\times$. Increasing the spill interval c from 16 to 32 yields an additional 1.02 $\times$ to 1.05 $\times$ improvement. Even for long requests with the 175B model, our system supports over 4.08 million requests.

Energy Efficiency. Figure 17(a) presents the energy consumption breakdown. We measure GPU power using NVML [99] and CPU/DRAM power using Intel RAPL [147]. The power for SmartSSDs is measured from its PCIe expansion board controller [12] feature [48]. For the baseline PCIe 4.0 SSD, we use its datasheet value of 13 W [121]. Although HILOS's SmartSSDs consume more power than conventional SSDs, the significant reduction in inference latency yields superior energy efficiency. The FLEX(SSD) baseline exhibits poor efficiency due to low throughput. While FLEX(DRAM) is efficient for the 30B and 66B models, its efficiency degrades for the 175B model due to batch size limitations.

Multi-Node Solutions. In Figure 17(b), we compared HILOS with a distributed multi-GPU configuration employing vLLM 0.9.1, which incorporates paged attention [71] and FlashAttention [30]. The system consists of two nodes utilizing tensor parallelism [130] within each node and pipeline

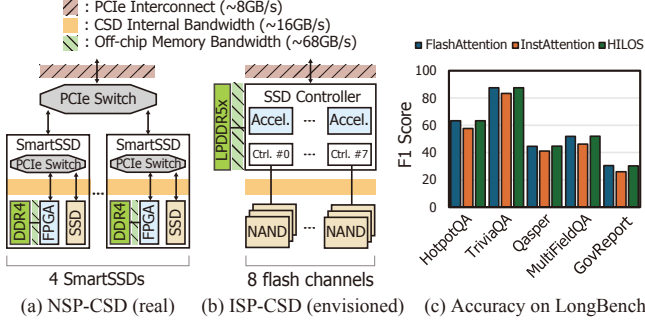


Figure 18. (a) Our NSP-CSD prototype, which effectively emulates (b) an envisioned ISP-CSD. (c) F1 on LongBench [15].

parallelism [54, 56] across them. Each node is equipped with four 48 GB RTX A6000 GPUs, 512 GB of host memory, an AMD EPYC 7302 CPU, and an InfiniBand EDR interconnect. HILOS achieves a 1.64× to 1.81× speedup over this multi-node baseline. The distributed setup is bottlenecked by small batches and inter-node communication overheads, despite having a large aggregated GPU memory of 384 GB (8×48GB). While techniques like context parallelism [84, 85] can efficiently parallelize long sequences, they do not solve the fundamental problem of fitting the large model into GPU memory. Consequently, supporting both long contexts and >100B parameter LLMs with context and model parallelism requires additional servers, which increases inter-server communication overhead, deployment cost [86], and power [134].

7 Discussion

7.1 Applicability to ISP Solutions.

In this paper, we developed HILOS on an NSP environment [9]. Although this is a solution that can be used today or in the near future, it has a limitation in that a single NSP device does not increase the internal bandwidth over a conventional system. Fortunately, we envision that the core techniques used in HILOS can also be applied to future In-Storage Processing (ISP) solutions [41, 89, 100, 146, 158], which provide considerable bandwidth improvements with a single device. Figure 18(b), illustrates an envisioned HILOS-enabled ISP device, modeled after modern SSD designs [93, 121, 123]. The NAND flash array provides an aggregated capacity of 16 TB and is connected to the SSD controller via eight flash channels, each providing 2,000MT/s (total of 16 GB/s). The single-package LPDDR5X (with four 16GB channels) as an internal DRAM offers an aggregated bandwidth of 68 GB/s.

From this, we argue that the performance benefit of this single PCIe 4.0 ISP unit would closely match that of the four SmartSSDs we used, in terms of the bandwidths of the internal storage (16 GB/s for 16 internal PCIe 3.0 lanes), system interconnect (8 GB/s for four PCIe 4.0 lanes), and the internal memory (52 GB/s for four DDR4 channels [9]). Even though SmartSSDs are unable to fully emulate the envisioned

ISP devices due to restricted firmware access, we believe they provide a reasonable proxy, given highly regular KV access patterns simplify SSD-internal managements [41, 67, 89].

We evaluate the overhead of our proposed accelerator for ISP setup by synthesizing its design with $d_{group} = 1$ using the OpenROAD Flow [5, 6], an open-source RTL-to-GDSII toolchain with the 45nm Nangate45 (FreePDK45) and scaled to an 8 nm process node to reflect modern SSD controllers' technology node [121] clocked at 300MHz (to match our FPGA configuration). Along with the on-chip SRAM modeled using CACTI 7.0 [16], the total chip area is 0.47 mm², with power at 1.13 W on a 32K-token inference profile, which we believe is a reasonable overhead for ISP.

Attention Offloading in ISP. The concept of offloading attention operations to ISP is explored in InstAttention [100]. However, to meet the resource constraints of ISP, it adopts sparse KV retrieval techniques [77, 77, 100, 116, 138, 161, 163] with lossy compression. While this method preserves accuracy for short context models [143, 162] up to 4K tokens, we observe noticeable accuracy drops for modern models with long contexts exceeding 32K tokens [37, 92, 139, 140]. To quantify this degradation, we evaluated Qwen2.5-32B-32K [140] on five LongBench [15] datasets in Figure 18(c). With InstAttention's default 1/8 compression ratio, the accuracy degrades by 3.52 ~ 5.73%. Unlike lossy schemes, the HILOS accelerator (§ 4.4) maintains lossless accuracy compared to the widely used FlashAttention [30].

It is also worth noting that while InstAttention's attention offloading provides valuable insights, its practicality remains limited due to its implementation on a relatively outdated OpenSSD research platform [26, 70] with emulation-based evaluation [68] for multi-SSD configurations. In contrast, HILOS is thoroughly studied on a modern hardware platform and addresses the practical issues that arise when attention offloading is employed in real NSP-enabled systems.

7.2 Future Computational Storage Device Designs

Despite the architectural optimizations in § 4.4 and § 5.4, there remain several limitations posed by the existing CSD platforms. In specific, while our current CSD design saturates the bandwidth of PCIe 3.0 SSDs, future PCIe 5.0 devices will require a fourfold increase in throughput to fully utilize storage I/O, which would make it more challenging for CSD-based systems to maintain speedups. In such regard, we analyze current hardware limitations and explore architectural refinements to meet future demands.

First, softmax operations dominate execution time as the attention group size (d_{group}) increases. It accounts for over 50% of total execution time and consumes the majority of DSP resources due to computationally intensive floating-point exponential operations. To match a 4× throughput increase from the assumed PCIe 5.0 interface via DSP parallelization, the design would require over 2,000 DSPs, which exceeds the total capacity of current SmartSSD platforms.

One solution we believe to be reasonable is to provide dedicated units for exponential functions. Given the prevalence of exponential functions in modern AI workloads such as sigmoid and GeLU, dedicated hardware units for them would significantly enhance the viability of CSDs for deep learning.

Second, the adoption of separate clock domains with HLS would greatly facilitate timing closure. Because contemporary HLS tools [10] enforce a single clock domain, they inherently limit the scope of advanced frequency optimization. Supporting distinct domains would allow the compute-intensive softmax logic to operate at high frequencies while maintaining lower frequencies for memory-bound GEMV logic. This decoupling would provide a favorable trade-off between throughput and the strict power constraints of NSP.

Finally, the multi-SSD distributed execution model of HILOS indicate that there might be a mismatch between existing hardware resources and actual operational requirements. For instance, while KV Caches are partitioned across multiple CSDs to maximize aggregate internal bandwidth, this distribution results in a low storage footprint on each individual device. Even under peak workloads, storage capacity overhead per SSD remains below 600 GB, leaving the 4 TB capacity of current SmartSSDs largely unexploited. Furthermore, while current CSDs match internal and external lane bandwidths (Figure 18(a)), most traffic remains internal, leaving host-to-CSD lanes underutilized. Higher internal bandwidth could be achieved through lightweight SSD mechanisms, such as coarse-grained block-level FTL mappings instead of DRAM-intensive page-level mappings [41, 67, 89]. This approach is particularly effective because our design ensures sequential KV Cache accesses for both reads and writes through the write-back mechanism in § 4.3. To this extent, a more balanced design could be viable, such as less capacity, more internal bandwidth, and/or more computational power per CSD. We believe such modifications would lead to a more cost-efficient system.

7.3 Applicability to CXL-based Architectures

The low-latency coherent memory access provided by Compute Express Link (CXL) [28] offers a significant potential to further reduce data management overhead in HILOS. Our current cache management (§ 4.3) employs host-side buffering followed by explicit transmission to the near-storage accelerator. This overhead scales with the spill interval (c) as shown in Figure 13. This bottleneck is expected to intensify as modern SSDs adopt larger internal page granularities like 16 KiB [120], which necessitates more efficient data handling.

Physical memory isolation in PCIe-based environments necessitates explicit DMA orchestration via Xilinx Runtime (XRT) between FPGA DRAM and host DRAM. This results in substantial software overhead and synchronization complexity [150], reducing throughput by over 30% when scaling c from 4 KiB ($c=16$) to 16 KiB ($c=64$), as illustrated in Figure 13.

CXL.mem can address these limitations by enabling a unified address space, which eliminates explicit data copies and DMA management, thereby reducing latency and simplifying the underlying cache management logic.

8 Related Work

8.1 LLM Inference Acceleration.

Prior work on LLM inference include kernel optimizations [11, 151], parameter quantization [33, 53, 87, 165], parallelism [85, 107], batching strategies [3, 27, 157], disaggregation [104, 167], and MoE considerations [20]. For limited GPU resources, many approaches adopt an offloading strategy for both training [14, 57, 114, 115] and inference [7, 61, 100, 127, 133, 158, 164]. HILOS shares a similar spirit but leverages NSP for offline batched inference across long sequences, targeting applications like large-scale information extraction [22, 44, 96, 101, 108] and benchmarking [46, 47, 81, 166].

8.2 Scalable Memory and Access

Memory Expansion. Emerging interconnects like CXL [28, 32, 43] and NVLink-C2C [38, 148] address memory capacity limitations. However, DRAM costs remain significantly higher than flash (approximately \$3 [122] vs \$0.1 [121] per GB [24]), and scaling DRAM to accommodate the growing KV cache demands of long-context [37, 83, 92, 139, 140] and multi-turn models [39, 59] is challenging. Although CXL-based near-data processing [58, 66, 103, 156] shares design philosophies with our approach, HILOS specifically targets PCIe-based storage environments. Nevertheless, our principles of minimizing host-device traffic, enabling cooperative host-device processing, and ensuring lightweight accelerator design remain applicable to the CXL-based systems.

Direct Memory Access. To minimize data management overheads, orchestrated DMA techniques [21, 100, 112, 150] allow accelerators to initiate storage I/O directly. This mitigates the inefficiency of host-managed schemes [17, 98, 160] where the CPU manages NVMe interactions. HILOS aligns with this design principle to alleviate CPU bottlenecks. Specifically, we offload the memory-intensive multi-head attention tasks to the near-storage accelerator, thereby effectively eliminating CPU-bound performance limitations.

8.3 Near-Data Processing

Processing In Memory (PIM). While PIM architectures typically integrate computation within DRAM [49, 75, 76] and have seen commercial realization [34, 75, 76], recent works primarily apply DRAM-PIM [65, 75, 76, 128, 129] to LLM inference [50, 72, 80, 102, 103, 126, 159, 168].

Near-Storage Processing (NSP). Early NSP research focused on “active” disks [1, 63, 117, 118], followed by embedded CPUs [62, 111] or integrated processors [13, 45]. Limitations in compute power led to the adoption of ASICs [82,

88, 89] and FPGAs [74, 131, 137, 149, 155], eventually resulting in commercial platforms [9] applied to diverse domains [35, 57, 78, 132, 141]. We distinguish HILOS from prior NSP efforts that target FFN offloading [61] or optimize for on-device low-batch scenarios [73, 136, 158]. Specifically, HILOS targets KV-cache I/O to maximize throughput for batched offline inference. By addressing DRAM capacity bottlenecks unique to this scenario, HILOS achieves improved system-level throughput compared to prior architectures.

9 Conclusion

We present HILOS, an NSP solution that accelerates offline LLM inference by offloading KV cache-related operations to near-storage accelerators. By mitigating critical system-level integration challenges and employing a specialized hardware design, HILOS achieves up to 7.86× higher throughput and 85% energy reduction over state-of-the-art baselines in real system evaluations using a PyTorch-based inference pipeline.

Acknowledgments

This work was supported by Samsung Memory Research Center (SMRC), National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (2022R1C1C1011307, RS-2025-00519994), and Institute of Information & communications Technology Planning & Evaluation (IITP) (RS-2024-00395134, RS-2024-00347394, RS-2023-00256081, RS-2021-II211343, RS-2024-00459026). Part of the infrastructure used in this work was supported by Korea Basic Science Institute (National research Facilities and Equipment Center) grant funded by the Ministry of Science and ICT (No. RS-2025-00564840). Jinho Lee is the corresponding author.

A Artifact Appendix

A.1 Abstract

This artifact provides the source code and hardware configuration for HILOS, a high-throughput NSP system optimized for long-context LLM offline inference. It comprises: (1) HLS source code for FPGA-based attention acceleration kernels and (2) host-side system software. The artifact supports both the end-to-end synthesis of FPGA binaries and the deployment of the full inference pipeline.

A.2 Description

A.2.1 How to Access. The source code is permanently archived via Zenodo at <https://doi.org/10.5281/zenodo.18162119>. The latest development version is maintained at <https://github.com/hongsunjang/HILOS/tree/asplos26>.

A.2.2 Hardware dependencies. Reproducing the experiments requires the following hardware configuration.

- **GPU:** At least one NVIDIA GPU with GPUDirect Storage (GDS) support.

- **Storage:** At least one Samsung SmartSSD.
- **CPU:** An Intel CPU is mandatory for SmartSSD compatibility.

A.2.3 Software dependencies. The environment relies on the following components.

- **Drivers and CUDA:** NVIDIA driver (version 530 or higher) and CUDA/CuDNN toolkit (version 12.1).
- **GPUDirect Storage (GDS):** The NVIDIA GDS package appropriate for the system kernel.
- **FPGA Tools:** Xilinx Vivado and Vitis HLS (version 2023.1).
- **Xilinx Runtime (XRT):** The XRT version matching the target SmartSSD platform. A cold reboot is mandatory after installation to flash the SmartSSD.

Source the settings scripts prior to execution.

```
$ source <path_to_Vitis_HLS>/2023.1/settings64.sh
$ source /opt/xilinx/xrt/setup.sh
```

A.3 Installation

For detailed, step-by-step instructions on environment setup and dependency installation, please refer to the README.md file in the root directory of the repository: <https://github.com/hongsunjang/HILOS/tree/asplos26>

A.4 Experiment Workflow

The evaluation follows a two-step workflow involving (1) FPGA binary synthesis and (2) LLM inference deployment.

A.4.1 Step 1: HLS Synthesis and Verification.

1. **Verify HLS Functionality:** Run the test script to validate the HLS implementation.
\$ python tests/test_llm.py --mode hls_gqa
 2. **Verify Binary Synthesis:** Copy the generated bitstream to the bins folder and verify functionality.
\$ python tests/test_llm.py --mode fpga_gqa
- Verify that the token output matches the expected values and that the kernel executes without errors.

A.4.2 Step 2: LLM Inference Deployment.

1. **Proposed Method (ANS):** Execute the system with FPGA acceleration.
\$ python3 bench_suite.py hilos
2. **Proposed Method (+X-Cache):** Execute HILOS combined with the X-Cache optimization.
\$ python3 bench_suite.py xcach

A.5 Evaluation and Expected Results

The benchmark script outputs generated tokens and decoding throughput. Verify token correctness by confirming that the baseline and HILOS produce identical tokens. Assess performance by comparing the decoding throughput, displayed as decode throughput: <val> token/s.

References

- [1] Anurag Acharya, Mustafa Uysal, and Joel Saltz. 1998. Active Disks: Programming Model, Algorithms and Evaluation. In *Proceedings of the Eighth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*.
- [2] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. GPT-4 Technical Report. *arXiv preprint arXiv:2303.08774* (2023).
- [3] Amey Agrawal, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S Gulavani, and Ramachandran Ramjee. 2023. Sarathi: Efficient LLM Inference by Piggybacking Decodes with Chunked Prefills. *arXiv preprint arXiv:2308.16369* (2023).
- [4] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. 2023. GQA: Training Generalized Multi-query Transformer Models from Multi-head Checkpoints. *arXiv preprint arXiv:2305.13245* (2023).
- [5] T Ajayi, D Blaauw, TB Chan, CK Cheng, VA Chhabria, DK Choo, M Coltella, S Dobre, R Dreslinski, M Fogaça, et al. 2019. OpenROAD: Toward a Self-Driving, Open-Source Digital Layout Implementation Tool Chain. In *Proceedings of Government Microcircuit Applications and Critical Technology Conference (GOMACTech)*.
- [6] Tutu Ajayi, Vidya A. Chhabria, Mateus Fogaça, Soheil Hashemi, Abdelrahman Hosny, Andrew B. Kahng, Minsoo Kim, Jeongsup Lee, Uday Mallappa, Marina Neseem, Geraldo Pradipta, Sherief Reda, Mehdi Saligane, Sachin S. Sapatnekar, Carl Sechen, Mohamed Shalan, William Swartz, Lutong Wang, Zhehong Wang, Mingyu Woo, and Bangqi Xu. 2019. Toward an Open-Source Digital Flow: First Learnings from the OpenROAD Project. In *Proceedings of the 56th Annual Design Automation Conference (DAC)*.
- [7] Keivan Alizadeh, Iman Mirzadeh, Dmitry Belenko, Karen Khatamifard, Minsik Cho, Carlo C Del Mundo, Mohammad Rastegari, and Mehrdad Farajtabar. 2023. Llm in a Flash: Efficient Large Language Model Inference with Limited Memory. *arXiv preprint arXiv:2312.11514* (2023).
- [8] Tyler Allen and Rong Ge. 2021. In-depth Analyses of Unified Virtual Memory System for GPU Accelerated Computing. In *SC'21: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*.
- [9] AMD/Samsung. [n. d.]. SmartSSD. <https://www.xilinx.com/publications/product-briefs/xilinx-smartssd-computational-storage-drive-product-brief.pdf>. Accessed: 2025-12-12.
- [10] AMD/Xilinx. 2021. Vitis High-Level Synthesis User Guide (ug1399).
- [11] Reza Yazdani Aminabadi, Samyam Rajbhandari, Ammar Ahmad Awan, Cheng Li, Du Li, Elton Zheng, Olatunji Ruwase, Shaden Smith, Minjia Zhang, Jeff Rasley, and Yuxiong He. 2022. DeepSpeed-Inference: Enabling Efficient Inference of Transformer Models at Unprecedented Scale. In *SC'22: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*.
- [12] ASPEED. [n. d.]. AST2500/AST2520. https://vgamuseum.info/images/doc/aspeed/ast2520a2gpg_datasheet.pdf. Accessed: 2025-12-12.
- [13] Duck-Ho Bae, Jin-Hyung Kim, Sang-Wook Kim, Hyunok Oh, and Chanik Park. 2013. Intelligent SSD: A Turbo for Big Data Mining. In *Proceedings of the 22nd ACM International Conference on Information & Knowledge Management (CIKM)*.
- [14] Jonghyun Bae, Jongsung Lee, Yunho Jin, Sam Son, Shine Kim, Hakbeom Jang, Tae Jun Ham, and Jae W. Lee. 2021. FlashNeuron: SSD-Enabled Large-Batch Training of Very Deep Neural Networks. In *Proceedings of the 19th USENIX Conference on File and Storage Technologies (FAST)*.
- [15] Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, et al. 2024. Longbench: A Bilingual, Multitask Benchmark for Long Context Understanding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*.
- [16] Rajeev Balasubramanian, Andrew B. Kahng, Naveen Muralimanohar, Ali Shafiee, and Vaishnav Srinivas. 2017. CACTI 7: New Tools for Interconnect Exploration in Innovative Off-Chip Memories. *ACM Trans. Archit. Code Optim.* 14, 2, Article 14 (2017), 25 pages.
- [17] Shai Bergman, Tanya Brokhman, Tzachi Cohen, and Mark Silberstein. 2019. SPIN: Seamless Operating System Integration of Peer-to-Peer DMA between SSDs and GPUs. *ACM Trans. Comput. Syst.* 36, 2, Article 5 (2019), 26 pages.
- [18] Alexander Borzunov, Dmitry Baranchuk, Tim Dettmers, Max Ryabinin, Younes Belkada, Artem Chumachenko, Pavel Samygin, and Colin Raffel. 2022. Petals: Collaborative Inference and Fine-Tuning of Large Models. *arXiv preprint arXiv:2209.01188* (2022).
- [19] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. In *Proceedings of the Advances in Neural Information Processing Systems 33 (NeurIPS)*.
- [20] Shiyi Cao, Shu Liu, Tyler Griggs, Peter Schafhalter, Xiaoxuan Liu, Ying Sheng, Joseph E. Gonzalez, Matei Zaharia, and Ion Stoica. 2025. MoE-Lightning: High-Throughput MoE Inference on Memory-Constrained GPUs. In *Proceedings of the 30th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*.
- [21] Chia-Hao Chang, Jihoon Han, Anand Sivasubramaniam, Vikram Sharma Mailthody, Zaid Qureshi, and Wen-Mei Hwu. 2024. GMT: GPU Orchestrated Memory Tiering for the Big Data Era. In *Proceedings of the 29th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*.
- [22] Yapei Chang, Kyle Lo, Tanya Goyal, and Mohit Iyyer. 2024. BoookScore: A Systematic Exploration of Book-Length Summarization in the Era of LLMs. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*.
- [23] Hongzheng Chen, Jiahao Zhang, Yixiao Du, Shaojie Xiang, Zichao Yue, Niansong Zhang, Yaohui Cai, and Zhiru Zhang. 2024. Understanding the Potential of FPGA-based Spatial Acceleration for Large Language Model Inference. *ACM Trans. Reconfigurable Technol. Syst.* 18, 1, Article 5 (2024), 29 pages.
- [24] Kangqi Chen, Rakesh Nadig, Manos Frouzakakis, Nika Mansouri Ghiasi, Yu Liang, Haiyu Mao, Jisung Park, Mohammad Sadrosadati, and Onur Mutlu. 2025. REIS: A High-Performance and Energy-Efficient Retrieval System with In-Storage Processing. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA)*.
- [25] Yinfang Chen, Huaibing Xie, Minghua Ma, Yu Kang, Xin Gao, Liu Shi, Yunjie Cao, Xuedong Gao, Hao Fan, Ming Wen, Jun Zeng, Supriyo Ghosh, Xuchao Zhang, Chaoyun Zhang, Qingwei Lin, Saravan Rajmohan, Dongmei Zhang, and Tianyin Xu. 2024. Automatic Root Cause Analysis via Large Language Models for Cloud Incidents. In *Proceedings of the Nineteenth European Conference on Computer Systems (EuroSys)*.
- [26] CRZ Technology Inc. [n. d.]. Daisplus Openssd. <https://www.crz-tech.com/crz/article/DaisyPlus/>. Accessed: 2025-12-12.
- [27] Weihao Cui, Han Zhao, Quan Chen, Hao Wei, Zirui Li, Deze Zeng, Chao Li, and Minyi Guo. 2022. DVABatch: Diversity-aware Multi-Entry Multi-Exit Batching for Efficient Processing of DNN Services on GPUs. In *Proceedings of the 2022 USENIX Annual Technical Conference (USENIX ATC)*.
- [28] CXL Consortium. [n. d.]. Compute Express Link. <https://www.computeexpresslink.org>. Accessed: 2025-12-12.

- [29] Zihang Dai, Zhilin Yang, Yiming Yang, Jaime Carbonell, Quoc Le, and Ruslan Salakhutdinov. 2019. Transformer-XL: Attentive Language Models beyond a Fixed-Length Context. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*.
- [30] Tri Dao. 2024. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*.
- [31] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. In *Proceedings of the Advances in Neural Information Processing Systems 35 (NeurIPS)*.
- [32] Debendra Das Sharma, Robert Blankenship, and Daniel Berger. 2024. An Introduction to the Compute Express Link (CXL) Interconnect. *ACM Comput. Surv.* 56, 11, Article 290 (2024), 37 pages.
- [33] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. 2022. GPT3.int8(): 8-bit Matrix Multiplication for Transformers at Scale. In *Proceedings of the Advances in Neural Information Processing Systems 35 (NeurIPS)*.
- [34] Fabrice Devaux. 2019. The True Processing in Memory Accelerator. In *Proceedings of the IEEE Hot Chips 31 Symposium (HCS)*.
- [35] Jaeyoung Do, Yang-Suk Kee, Jignesh M. Patel, Chanik Park, Kwanghyun Park, and David J. DeWitt. 2013. Query Processing on Smart SSDs: Opportunities and Challenges. In *Proceedings of the 2013 SIGMOD International Conference on Management of Data (SIGMOD)*.
- [36] Nan Du, Yanping Huang, Andrew M Dai, Simon Tong, Dmitry Lepikhin, Yuanzhong Xu, Maxim Krikun, Yanqi Zhou, Adams Wei Yu, Orhan Firat, Barret Zoph, Liam Fedus, Maarten P Bosma, Zongwei Zhou, Tao Wang, Emma Wang, Kellie Webster, Marie Pellat, Kevin Robinson, Kathleen Meier-Hellstern, Toju Duke, Lucas Dixon, Kun Zhang, Quoc Le, Yonghui Wu, Zhifeng Chen, and Claire Cui. 2022. GLaM: Efficient Scaling of Language Models with Mixture-of-Experts. In *Proceedings of the 39th International Conference on Machine Learning (ICML)*.
- [37] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. 2024. The Llama 3 Herd of Models. *arXiv preprint arXiv:2407.21783* (2024).
- [38] Anne C. Elster and Tor A. Haugdahl. 2022. Nvidia Hopper GPU and Grace CPU Highlights. *Computing in Science & Engineering* 24, 2 (2022), 95–100.
- [39] Bin Gao, Zhuomin He, Puru Sharma, Qingxuan Kang, Djordje Jevdjic, Junbo Deng, Xingkun Yang, Zhou Yu, and Pengfei Zuo. 2024. Cost-Efficient Large Language Model Serving for Multi-turn Conversations with CachedAttention. In *Proceedings of the 2024 USENIX Annual Technical Conference (USENIX ATC)*.
- [40] Leo Gao, Jonathan Tow, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Kyle McDonell, Niklas Muennighoff, et al. 2021. A Framework for Few-Shot Language Model Evaluation. *Zenodo* (2021).
- [41] Nika Mansouri Ghiasi, Mohammad Sadrosadati, Harun Mustafa, Arvid Gollwitzer, Can Firtina, Julien Eudine, Haiyu Mao, Joël Lindegger, Meryem Banu Cavlak, Mohammed Alser, Jisung Park, and Onur Mutlu. 2024. MegIS: High-Performance, Energy-Efficient, and Low-Cost Metagenomic Analysis with In-Storage Processing. In *Proceedings of the 51st Annual International Symposium on Computer Architecture (ISCA)*.
- [42] Github. [n. d.]. Github Copilot. <https://github.com/features/copilot/>. Accessed: 2025-12-12.
- [43] Donghyun Gouk, Miryeong Kwon, Hanyeoreum Bae, Sangwon Lee, and Myoungsoo Jung. 2023. Memory Pooling With CXL. *IEEE Micro* 43, 2 (2023), 48–57.
- [44] Tanya Goyal, Junyi Jessy Li, and Greg Durrett. 2022. News Summarization and Evaluation in the Era of Gpt-3. *arXiv preprint arXiv:2209.12356* (2022).
- [45] Boncheol Gu, Andre S. Yoon, Duck-Ho Bae, Insoon Jo, Jinyoung Lee, Jonghyun Yoon, Jeong-Uk Kang, Moonsang Kwon, Chanhoo Yoon, Sangyeun Cho, Jaeheon Jeong, and Duckhyun Chang. 2016. Biscuit: A Framework for Near-Data Processing of Big Data Workloads. In *Proceedings of the 43rd International Symposium on Computer Architecture (ISCA)*.
- [46] Neel Guha, Julian Nyarko, Daniel Ho, Christopher Ré, Adam Chilton, Aditya K, Alex Chohlas-Wood, Austin Peters, Brandon Waldon, Daniel Rockmore, Diego Zambrano, Dmitry Talisman, Enam Hoque, Faiz Surani, Frank Fagan, Galit Sarfaty, Gregory Dickinson, Haggai Porat, Jason Hegland, Jessica Wu, Joe Nudell, Joel Niklaus, John Nay, Jonathan Choi, Kevin Tobia, Margaret Hagan, Megan Ma, Michael Livermore, Nikon Rasumov-Rahe, Nils Holzenberger, Noam Kolt, Peter Henderson, Sean Rehaag, Sharad Goel, Shang Gao, Spencer Williams, Sunny Gandhi, Tom Zur, Varun Iyer, and Zehua Li. 2023. LegalBench: A Collaboratively Built Benchmark for Measuring Legal Reasoning in Large Language Models. In *Proceedings of the Advances in Neural Information Processing Systems 36 (NeurIPS)*.
- [47] Jiayan Guo, Lun Du, and Hengyu Liu. 2023. Gpt4graph: Can Large Language Models Understand Graph Structured Data? An Empirical Evaluation and Benchmarking. *arXiv preprint arXiv:2305.15066* (2023).
- [48] H3. [n. d.]. Falcon 4109. <https://www.h3platform.com/product-detail/overview/25>. Accessed: 2025-12-12.
- [49] Mingxuan He, Choungki Song, Ilkon Kim, Chunseok Jeong, Seho Kim, Il Park, Mithuna Thottethodi, and T. N. Vijaykumar. 2020. Newton: A DRAM-maker's Accelerator-in-Memory (AiM) Architecture for Machine Learning. In *Proceedings of the 53th Annual International Symposium on Microarchitecture (MICRO)*.
- [50] Guseul Heo, Sangyeop Lee, Jaehong Cho, Hyunmin Choi, Sanghyeon Lee, Hyungkyu Ham, Gwangsun Kim, Divya Mahajan, and Jongse Park. 2024. NeuPIMs: NPU-PIM Heterogeneous Acceleration for Batched LLM Inferencing. In *Proceedings of the 29th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*.
- [51] Connor Holmes, Masahiro Tanaka, Michael Wyatt, Ammar Ahmad Awan, Jeff Rasley, Samyam Rajbhandari, Reza Yazdani Aminabadi, Heyang Qin, Arash Bakhtiari, Lev Kurilenko, et al. 2024. Deepspeed-Fastgen: High-Throughput Text Generation for LLMs via MII and Deepspeed-Inference. *arXiv preprint arXiv:2401.08671* (2024).
- [52] Seongmin Hong, Seungjae Moon, Junsoo Kim, Sungjae Lee, Minsub Kim, Dongsoo Lee, and Joo-Young Kim. 2022. DFX: A Low-latency Multi-FPGA Appliance for Accelerating Transformer-based Text Generation. In *Proceedings of the 55th Annual International Symposium on Microarchitecture (MICRO)*.
- [53] Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W Mahoney, Yakun Sophia Shao, Kurt Keutzer, and Amir Gholami. 2024. KVQuant: Towards 10 Million Context Length LLM Inference with KV Cache Quantization. *arXiv preprint arXiv:2401.18079* (2024).
- [54] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Dehao Chen, Mia Chen, HyoukJoong Lee, Jiquan Ngiam, Quoc V Le, Yonghui Wu, and zhifeng Chen. 2019. GPipe: Efficient Training of Giant Neural Networks using Pipeline Parallelism. In *Proceedings of the Advances in Neural Information Processing Systems 32 (NeurIPS)*.
- [55] Hugging Face. [n. d.]. Hugging Face. <https://huggingface.co/docs/hub/index>. Accessed: 2025-12-12.
- [56] Hongsun Jang, Jaewon Jung, Jaeyong Song, Joonsang Yu, Youngsok Kim, and Jinho Lee. 2023. Pipe-BD: Pipelined Parallel Blockwise Distillation. In *Proceedings of the 2023 Design, Automation & Test in Europe Conference & Exhibition (DATE)*.
- [57] Hongsun Jang, Jaeyong Song, Jaewon Jung, Jaeyoung Park, Youngsok Kim, and Jinho Lee. 2024. Smart-Infinity: Fast Large Language Model Training using Near-Storage Processing on a Real System. In *Proceedings of the 2024 International Symposium on High-Performance*

Computer Architecture (HPCA).

- [58] Junhyeok Jang, Hanjin Choi, Hanyeoreum Bae, Seungjun Lee, Miryeong Kwon, and Myoungsoo Jung. 2023. CXL-ANNS: Software-Hardware Collaborative Memory Disaggregation and Computation for Billion-Scale Approximate Nearest Neighbor Search. In *Proceedings of the 2023 USENIX Annual Technical Conference (USENIX ATC)*.
- [59] Jinwoo Jeong and Jeongseob Ahn. 2025. Accelerating LLM Serving for Multi-turn Dialogues with Efficient Resource Management. In *Proceedings of the 30th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*.
- [60] Albert Q Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mesch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al. 2024. Mixtral of Experts. *arXiv preprint arXiv:2401.04088* (2024).
- [61] Yunho Jin, Shine Kim, Tae Jun Ham, and Jae W. Lee. 2022. Architecting a Flash-Based Storage System for Low-Cost Inference of Extreme-Scale DNNs. *IEEE Trans. Comput.* (2022).
- [62] Yangwook Kang, Yang-suk Kee, Ethan L. Miller, and Chanik Park. 2013. Enabling Cost-Effective Data Processing with Smart SSD. In *Proceedings of the 29th Symposium on Mass Storage Systems and Technologies (MSST)*.
- [63] Kimberly Keeton, David A. Patterson, and Joseph M. Hellerstein. 1998. A Case for Intelligent Disks (IDISks). *SIGMOD Rec.* 27, 3 (1998), 42–52.
- [64] Hamza Khan, Asma Khan, Zainab Khan, Lun Bin Huang, Kun Wang, and Lei He. 2021. NPE: An FPGA-based Overlay Processor for Natural Language Processing. In *Proceedings of the 2021 International Symposium on Field-Programmable Gate Arrays (FPGA)*.
- [65] Heesu Kim, Hanmin Park, Taehyun Kim, Kwanheum Cho, Eojin Lee, Soojung Ryu, Hyuk-Jae Lee, Kiyoun Choi, and Jinho Lee. 2021. GradPIM: A practical processing-in-DRAM architecture for gradient descent. In *Proceedings of the 2021 International Symposium on High-Performance Computer Architecture (HPCA)*.
- [66] Hyungyo Kim, Nachuan Wang, Qirong Xia, Jinghan Huang, Amir Yazdanbakhsh, and Nam Sung Kim. 2025. LIA: A Single-GPU LLM Inference Acceleration with Cooperative AMX-Enabled CPU-GPU Computation and CXL Offloading. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA)*.
- [67] Myungsuk Kim, Jaehoon Lee, Sungjin Lee, Jisung Park, and Jihong Kim. 2017. Improving Performance and Lifetime of Large-Page NAND Storages Using Erase-Free Subpage Programming. In *Proceedings of the 54th Annual Design Automation Conference (DAC)*.
- [68] Sang-Hoon Kim, Jaehoon Shim, Euidong Lee, Seongyeop Jeong, Ilkueon Kang, and Jin-Soo Kim. 2023. NVMeVirt: A Versatile Software-defined Virtual NVMe Device. In *Proceedings of the 21st USENIX Conference on File and Storage Technologies (FAST)*.
- [69] P. Kumar, S. Manikandan, and R. Kishore. 2024. AI-Driven Text Generation: A Novel GPT-Based Approach for Automated Content Creation. In *Proceedings of the 2024 International Conference on Networking and Communications (ICNWC)*.
- [70] Jaewook Kwak, Sangjin Lee, Kibin Park, Jinwoo Jeong, and Yong Ho Song. 2020. Cosmos+ OpenSSD: Rapid Prototype for Flash Storage Systems. *ACM Trans. Storage* 16, 3, Article 15 (2020), 35 pages.
- [71] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*.
- [72] Hyojung Lee, Daehyeon Baek, Jinyoung Son, Jieun Choi, Kihyo Moon, and Minsung Jang. 2025. PAISE: PIM-Accelerated Inference Scheduling Engine for Transformer-based LLM. In *Proceedings of the 2025 International Symposium on High Performance Computer Architecture (HPCA)*.
- [73] Jaeyong Lee, Hyeunjo Kim, Sanghun Oh, Myoungjun Chun, Myungsuk Kim, and Jihong Kim. 2025. AiF: Accelerating On-Device LLM Inference Using In-Flash Processing. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA)*.
- [74] Jinho Lee, Heesu Kim, Sungjoo Yoo, Kiyoun Choi, H. Peter Hofstee, Gi-Joon Nam, Mark R. Nutter, and Damir Jamsek. 2017. ExtraV: Boosting Graph Processing near Storage with a Coherent Accelerator. In *Proceedings of the 43rd International Conference on Very Large Data Bases (VLDB)*.
- [75] Sukhan Lee, Shin-haeng Kang, Jaehoon Lee, Hyeonsu Kim, Eojin Lee, Seungwoo Seo, Hosang Yoon, Seungwon Lee, Kyoungwan Lim, Hyunsung Shin, Jinhyun Kim, O Seongil, Anand Iyer, David Wang, Kyomin Sohn, and Nam Sung Kim. 2021. Hardware Architecture and Software Stack for PIM Based on Commercial DRAM Technology : Industrial Product. In *Proceedings of the 48st Annual International Symposium on Computer Architecture (ISCA)*.
- [76] Seongju Lee, Kyuyoung Kim, Sanghoon Oh, Joonhong Park, Gimoon Hong, Dongyoon Ka, Kyudong Hwang, Jeongje Park, Kyeongpil Kang, Jungyeon Kim, et al. 2022. A 1ynm 1.25 V 8Gb, 16Gb/s/pin GDDR6-based accelerator-in-memory supporting 1TFLOPS MAC operation and various activation functions for deep-learning applications. In *Proceedings of the 2022 International Solid-State Circuits Conference (ISSCC)*.
- [77] Wonbeom Lee, Jungi Lee, Junghwan Seo, and Jaewoong Sim. 2024. InfiniGen: Efficient Generative Inference of Large Language Models with Dynamic KV Cache Management. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.
- [78] Yunjae Lee, Jinha Chung, and Minsoo Rhu. 2022. SmartSAGE: training large-scale graph neural networks using in-storage processing architectures. In *Proceedings of the 49th Annual International Symposium on Computer Architecture (ISCA)*.
- [79] Bingbing Li, Santosh Pandey, Haowen Fang, Yanjun Lyv, Ji Li, Jieyang Chen, Mimi Xie, Lipeng Wan, Hang Liu, and Caiwen Ding. 2020. FTRANS: Energy-Efficient Acceleration of Transformers using FPGA. In *Proceedings of the 25th International Symposium on Low Power Electronics and Design (ISLPED)*.
- [80] Cong Li, Zhe Zhou, Yang Wang, Fan Yang, Ting Cao, Mao Yang, Yun Liang, and Guangyu Sun. 2024. PIM-DL: Expanding the Applicability of Commodity DRAM-PIMs for Deep Learning via Algorithm-System Co-Optimization. In *Proceedings of the 29th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*.
- [81] Percy Liang, Rishi Bommasani, Tony Lee, Dimitris Tsipras, Dilara Soylu, Michihiro Yasunaga, Yian Zhang, Deepak Narayanan, Yuhuai Wu, Ananya Kumar, Benjamin Newman, Binhang Yuan, Bobby Yan, Ce Zhang, Christian Cosgrove, Christopher D Manning, Christopher Re, Diana Acosta-Navas, Drew A. Hudson, Eric Zelikman, Esin Durmus, Faisal Ladhak, Frieda Rong, Hongyu Ren, Huaxiu Yao, Jue WANG, Keshav Santhanam, Laurel Orr, Lucia Zheng, Mert Yuksekgonul, Mirac Suzgun, Nathan Kim, Neel Guha, Niladri S. Chatterji, Omar Khattab, Peter Henderson, Qian Huang, Ryan Andrew Chi, Sang Michael Xie, Shibani Santurkar, Surya Ganguli, Tatsunori Hashimoto, Thomas Icard, Tianyi Zhang, Vishrav Chaudhary, William Wang, Xuechen Li, Yifan Mai, Yuhui Zhang, and Yuta Koreeda. 2023. Holistic Evaluation of Language Models. *Transactions on Machine Learning Research* (2023).
- [82] Jilan Lin, Ling Liang, Zheng Qu, Ishtiyaque Ahmad, Liu Liu, Fengbin Tu, Trinabh Gupta, Yufei Ding, and Yuan Xie. 2022. INSPIRE: In-Storage Private Information Retrieval via Protocol and Architecture Co-Design. In *Proceedings of the 49th Annual International Symposium on Computer Architecture (ISCA)*.
- [83] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. 2024. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437* (2024).

- [84] Hao Liu and Pieter Abbeel. 2023. Blockwise Parallel Transformers for Large Context Models. In *Proceedings of the Advances in Neural Information Processing Systems 36 (NeurIPS)*.
- [85] Hao Liu, Matei Zaharia, and Pieter Abbeel. 2024. RingAttention with Blockwise Transformers for Near-Infinite Context. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*.
- [86] Lian Liu, Shixin Zhao, Bing Li, Haimeng Ren, Zhaohui Xu, Mengdi Wang, Xiaowei Li, Yinhe Han, and Ying Wang. 2025. Make LLM Inference Affordable to Everyone: Augmenting GPU Memory with NDP-DIMM. In *Proceedings of the 2025 International Symposium on High Performance Computer Architecture (HPCA)*.
- [87] Zechun Liu, Barlas Oguz, Changsheng Zhao, Ernie Chang, Pierre Stock, Yashar Mehdad, Yangyang Shi, Raghuraman Krishnamoorthi, and Vikas Chandra. 2023. LLM-QAT: Data-free Quantization Aware Training for Large Language Models. *arXiv preprint arXiv:2305.17888* (2023).
- [88] Vikram Sharma Mailthody, Zaid Qureshi, Weixin Liang, Ziyan Feng, Simon Garcia De Gonzalo, Youjie Li, Hubertus Franke, Jinjun Xiong, Jian Huang, and Wen-mei Hwu. 2019. Deepstore: In-Storage Acceleration for Intelligent Queries. In *Proceedings of the 52nd Annual International Symposium on Microarchitecture (MICRO)*.
- [89] Nika Mansouri Ghiasi, Jisung Park, Harun Mustafa, Jeremie Kim, Ataberk Olgun, Arvid Gollwitzer, Damla Senol Cali, Can Firtina, Haiyu Mao, Nour Almadhoum Alser, Rachata Ausavarungnirun, Nandita Vijaykumar, Mohammed Alser, and Onur Mutlu. 2022. GenStore: A High-Performance In-Storage Processing System for Genome Sequence Analysis. In *Proceedings of the 27th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*.
- [90] Carver Mead and Lynn Conway. 1980. Introduction to VLSI systems.
- [91] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. 2017. Pointer Sentinel Mixture Models. In *Proceedings of the 5th International Conference on Learning Representations (ICLR)*.
- [92] Meta. [n. d.]. The Llama 4 herd: The beginning of a new era of natively multimodal AI innovation. Accessed: 2025-12-12.
- [93] Micron. [n. d.]. Crucial T500 1 TB. <https://www.techpowerup.com/ssd-specs/crucial-t500-1-tb.d1771>. Accessed: 2025-12-12.
- [94] Todor Mihaylov, Peter Clark, Tushar Khot, and Ashish Sabharwal. 2018. Can a Suit of Armor Conduct Electricity? A New Dataset for Open Book Question Answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.
- [95] Maxim Milakov and Natalia Gimelshein. 2018. Online Normalizer Calculation for Softmax. *arXiv preprint arXiv:1805.02867* (2018).
- [96] Shashi Narayan, Shay B Cohen, and Mirella Lapata. 2018. Don't Give Me the Details, Just the Summary! Topic-Aware Convolutional Neural Networks for Extreme Summarization. *arXiv preprint arXiv:1808.08745* (2018).
- [97] Neil Brown. [n. d.]. mdadm. <https://git.kernel.org/pub/scm/utils/mdadm/mdadm.git/>. Accessed: 2025-12-12.
- [98] NVIDIA. [n. d.]. GPUDirect Storage. <https://developer.nvidia.com/gpudirect-storage>. Accessed: 2025-12-12.
- [99] NVIDIA. [n. d.]. NVIDIA Management Library (NVML). <https://developer.nvidia.com/management-library-nvml>. Accessed: 2025-12-12.
- [100] Xiurui Pan, Endian Li, Qiao Li, Shengwen Liang, Yizhou Shan, Ke Zhou, Yingwei Luo, Xiaolin Wang, and Jie Zhang. 2025. InstAttention: In-Storage Attention Offloading for Cost-Effective Long-Context LLM Inference. In *Proceedings of the 2025 International Symposium on High-Performance Computer Architecture (HPCA)*.
- [101] Bo Pang, Erik Nijkamp, Wojciech Kryscinski, Silvio Savarese, Yingbo Zhou, and Caiming Xiong. 2023. Long Document Summarization with Top-down and Bottom-up Inference. In *Findings of the Association for Computational Linguistics (ACL Findings)*.
- [102] Jaehyun Park, Jaewan Choi, Kwanhee Kyung, Michael Jaemin Kim, Yongsuk Kwon, Nam Sung Kim, and Jung Ho Ahn. 2024. AttAcc! Unleashing the Power of PIM for Batched Transformer-based Generative Model Inference. In *Proceedings of the 29th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*.
- [103] Sang-Soo Park, KyungSoo Kim, Jinin So, Jin Jung, Jonggeon Lee, Kyounghwan Woo, Nayeon Kim, Younghyun Lee, Hyungyo Kim, Yongsuk Kwon, Jinhyun Kim, Jieun Lee, YeonGon Cho, Yongmin Tai, Jeonghyeon Cho, Hoyoung Song, Jung Ho Ahn, and Nam Sung Kim. 2024. An LPDDR-based CXL-PNM Platform for TCO-efficient Inference of Transformer-based Large Language Models. In *Proceedings of the 2024 International Symposium on High Performance Computer Architecture (HPCA)*.
- [104] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. In *Proceedings of the 51st Annual International Symposium on Computer Architecture (ISCA)*.
- [105] PCI-SIG. [n. d.]. PCI express base specification 4.0. <https://pcisig.com/specifications>. Accessed: 2025-12-12.
- [106] Lucian Petrica, Tobias Alonso, Mairin Kroes, Nicholas Fraser, Sorin Cotozana, and Michaela Blott. 2020. Memory-Efficient Dataflow Inference for Deep CNNs on FPGA. In *Proceedings of the 2020 International Conference on Field-Programmable Technology (ICFPT)*.
- [107] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. 2023. Efficiently scaling transformer inference. In *Proceedings of the Machine Learning and Systems 5 (MLSys)*.
- [108] Xiao Pu, Mingqi Gao, and Xiaojun Wan. 2023. Summarization is (Almost) Dead. *arXiv preprint arXiv:2309.09558* (2023).
- [109] pybind. [n. d.]. pybind11. <https://github.com/pybind/pybind11>. Accessed: 2025-12-12.
- [110] Python Software Foundation. [n. d.]. Distutils. <https://docs.python.org/3.9/library/distutils.html>. Accessed: 2025-12-12.
- [111] Luis Cavazos Quero, Young-Sik Lee, and Jin-Soo Kim. 2015. Self-Sorting SSD: Producing Sorted Data Inside Active SSDs. In *Proceedings of the 31th Symposium on Mass Storage Systems and Technologies (MSST)*.
- [112] Zaid Qureshi, Vikram Sharma Mailthody, Isaac Gelado, Seungwon Min, Amna Masood, Jeongmin Park, Jinjun Xiong, C. J. Newburn, Dmitri Vainbrand, I-Hsin Chung, Michael Garland, William Dally, and Wen-mei Hwu. 2023. GPU-Initiated On-Demand High-Throughput Storage Access in the BaM System Architecture. In *Proceedings of the 28th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*.
- [113] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. 2019. Language Models Are Unsupervised Multi-task Learners. *OpenAI blog* (2019).
- [114] Samyam Rajbhandari, Olatunji Ruwase, Jeff Rasley, Shaden Smith, and Yuxiong He. 2021. ZeRO-infinity: breaking the GPU memory wall for extreme scale deep learning. In *SC'21: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*.
- [115] Jie Ren, Samyam Rajbhandari, Reza Yazdani Aminabadi, Olatunji Ruwase, Shuangyan Yang, Minjia Zhang, Dong Li, and Yuxiong He. 2021. Zero-offload: democratizing billion-scale model training. In *Proceedings of the 2021 USENIX Annual Technical Conference (USENIX ATC)*.
- [116] Luka Ribar, Ivan Chelombiev, Luke Hudlass-Galley, Charlie Blake, Carlo Luschi, and Douglas Orr. 2024. SparQ Attention: Bandwidth-Efficient LLM Inference. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*.
- [117] E. Riedel, C. Faloutsos, G.A. Gibson, and D. Nagle. 2001. Active disks for large-scale data processing. *Computer* 34, 6 (2001), 68–74.

- [118] Erik Riedel, Garth A. Gibson, and Christos Faloutsos. 1998. Active Storage for Large-Scale Data Mining and Multimedia. In *Proceedings of the 24th International Conference on Very Large Data Bases (VLDB)*.
- [119] Melissa Roemmele, Cosmin Adrian Bejan, and Andrew S Gordon. 2011. Choice of Plausible Alternatives: An Evaluation of Commonsense Causal Reasoning. In *AAAI spring symposium: logical formalizations of commonsense reasoning*.
- [120] Mario Sako, Yoshihisa Watanabe, Takao Nakajima, Junpei Sato, Kazuyoshi Muraoka, Masaki Fujii, Fumihiko Kono, Michio Nakagawa, Masami Masuda, Koji Kato, Yuri Terada, Yuki Shimizu, Mitsuki Honma, Akihiro Imamoto, Tomoko Araya, Hayato Konno, Takuya Okanaga, Tomofumi Fujimura, Xiaoqing Wang, Mai Muramoto, Masahiro Kamoshida, Masatoshi Kohno, Yoshinao Suzuki, Tomoharu Hashiguchi, Tsukasa Kobayashi, Masashi Yamaoka, and Ryuji Yamashita. 2016. A Low Power 64 Gb MLC NAND-Flash Memory in 15 nm CMOS Technology. *IEEE Journal of Solid-State Circuits* 51, 1 (2016), 196–203.
- [121] Samsung. [n. d.]. PM9A3, 3.8TB. <https://www.techpowerup.com/ssd-specs/samsung-pm9a3-3-8-tb.d1255>. Accessed: 2025-12-12.
- [122] Samsung. [n. d.]. Samsung 128 GB DDR4 3200 LRDIMM ECC. <https://semiconductor.samsung.com/dram/module/lrdimm/m386aag40am3-cwe/>. Accessed: 2025-12-12.
- [123] Samsung. [n. d.]. Samsung 990 Pro 1 TB. <https://www.techpowerup.com/ssd-specs/samsung-990-pro-w-heatsink-1-tb.d1899>. Accessed: 2025-12-12.
- [124] Robert Schmid, Max Plauth, Lukas Wenzel, Felix Eberhardt, and Andreas Polze. 2020. Accessible Near-Storage Computing with FPGAs. In *Proceedings of the 15th European Conference on Computer Systems (EuroSys)*.
- [125] Yasar Seneviratne, Korakit Seemakhupt, Sihang Liu, and Samira Khan. 2023. NearPM: A Near-Data Processing System for Storage-Class Applications. In *Proceedings of the Eighteenth European Conference on Computer Systems (EuroSys)*.
- [126] Seong Hoon Seo, Junghoon Kim, Donghyun Lee, Seonah Yoo, Seokwon Moon, Yeonhong Park, and Jae W. Lee. 2025. FACIL: Flexible DRAM Address Mapping for SoC-PIM Cooperative On-device LLM Inference. In *Proceedings of the 2025 International Symposium on High Performance Computer Architecture (HPCA)*.
- [127] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Re, Ion Stoica, and Ce Zhang. 2023. FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*.
- [128] Changmin Shin, Jaeyong Song, Hongsun Jang, Dogeun Kim, Jun Sung, Taehee Kwon, Jae Hyung Ju, Frank Liu, Yeonkyu Choi, and Jinho Lee. 2025. Piccolo: Large-Scale Graph Processing with Fine-Grained In-Memory Scatter-Gather. In *Proceedings of the 2025 International Symposium on High Performance Computer Architecture (HPCA)*.
- [129] Changmin Shin, Jaeyong Song, Seongmin Na, Jun Sung, Hongsun Jang, and Jinho Lee. 2025. FALA: Locality-Aware PIM-Host Cooperation for Graph Processing with Fine-Grained Column Access. In *Proceedings of the 58th Annual International Symposium on Microarchitecture (MICRO)*.
- [130] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. Megatron-LM: Training Multi-Billion Parameter Language Models using Model Parallelism. *arXiv preprint arXiv:1909.08053* (2019).
- [131] Malcolm Singh and Ben Leonhardi. 2011. Introduction to the IBM Netezza warehouse appliance. In *Proceedings of the 2011 Conference of the Center for Advanced Studies on Collaborative Research (CASCON)*.
- [132] Mohammadreza Soltaniyeh, Veronica Lagrange Moutinho Dos Reis, Matt Bryson, Xuebin Yao, Richard P. Martin, and Santosh Nagarakatte. 2022. Near-Storage Processing for Solid State Drive Based Recommendation Inference with SmartSSDs®. In *Proceedings of the 2022 ACM/SPEC on International Conference on Performance Engineering (ICPE)*.
- [133] Yixin Song, Zeyu Mi, Haotong Xie, and Haibo Chen. 2023. Powerinfer: Fast large language model serving with a consumer-grade gpu. *arXiv preprint arXiv:2312.12456* (2023).
- [134] Jovan Stojkovic, Chaojie Zhang, Íñigo Goiri, Josep Torrellas, and Esha Choukse. 2025. DynamoLLM: Designing LLM Inference Clusters for Performance and Energy Efficiency. In *Proceedings of the 2025 International Symposium on High Performance Computer Architecture (HPCA)*.
- [135] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. 2024. Roformer: Enhanced Transformer with Rotary Position Embedding. *Neurocomputing* 568 (2024), 127063.
- [136] Weiye Sun, Mingyu Gao, Zhaoshi Li, Aoyang Zhang, Iris Ying Chou, Jianfeng Zhu, Shaojun Wei, and Leibo Liu. 2025. Lincoln: Real-Time 50 100B LLM Inference on Consumer Devices with LPDDR-Interfaced, Compute-Enabled Flash Memory. In *Proceedings of the 2025 International Symposium on High Performance Computer Architecture (HPCA)*.
- [137] Xuan Sun, Hu Wan, Qiao Li, Chia-Lin Yang, Tei-Wei Kuo, and Chun-Jason Xue. 2022. RM-SSD: In-Storage Computing for Large-Scale Recommendation Inference. In *Proceedings of the 2022 International Symposium on High-Performance Computer Architecture (HPCA)*.
- [138] Jiaming Tang, Yilong Zhao, Kan Zhu, Guangxuan Xiao, Baris Kasikci, and Song Han. 2024. QUEST: Query-Aware Sparsity for Efficient Long-Context LLM Inference. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*.
- [139] Gemini Team, Petko Georgiev, Ving Ian Lei, Ryan Burnell, Libin Bai, Anmol Gulati, Garrett Tanzer, Damien Vincent, Zhufeng Pan, Shibo Wang, et al. 2024. Gemini 1.5: Unlocking Multimodal Understanding Across Millions of Tokens of Context. *arXiv preprint arXiv:2403.05530* (2024).
- [140] Qwen Team. 2024. Qwen2 Technical Report. *arXiv preprint arXiv:2407.10671* (2024).
- [141] Bing Tian, Haikun Liu, Zhuohui Duan, Xiaofei Liao, Hai Jin, and Yu Zhang. 2024. Scalable Billion-point Approximate Nearest Neighbor Search Using SmartSSDs. In *Proceedings of the 2024 USENIX Annual Technical Conference (USENIX ATC)*.
- [142] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and Efficient Foundation Language Models. *arXiv preprint arXiv:2302.13971* (2023).
- [143] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shrutie Bhosale, et al. 2023. Llama 2: Open Foundation and Fine-tuned Chat Models. *arXiv preprint arXiv:2307.09288* (2023).
- [144] Yaman Umuroglu, Nicholas J. Fraser, Giulio Gambardella, Michaela Blott, Philip Leong, Magnus Jahre, and Kees Vissers. 2017. FINN: A Framework for Fast, Scalable Binarized Neural Network Inference. In *Proceedings of the 2017 International Symposium on Field-Programmable Gate Arrays (FPGA)*.
- [145] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is All You Need. In *Proceedings of the Advances in Neural Information Processing Systems 30 (NeurIPS)*.
- [146] Yuyue Wang, Xiurui Pan, Yuda An, Jie Zhang, and Glenn Reinman. 2024. BeaconGNN: Large-Scale GNN Acceleration with Out-of-Order Streaming In-Storage Computing. In *Proceedings of the 2024 International Symposium on High Performance Computer Architecture (HPCA)*.
- [147] Vincent M. Weaver, Matt Johnson, Kiran Kasichayanula, James Ralph, Piotr Luszczek, Dan Terpstra, and Shirley Moore. 2012. Measuring Energy and Power with PAPI. In *Proceedings of the 41st International Conference on Parallel Processing Workshops (ICPP Workshops)*.

- [148] Ying Wei, Yi Chieh Huang, Haiming Tang, Nithya Sankaran, Ish Chadha, Dai Dai, Olakanmi Oluwale, Vishnu Balan, and Edward Lee. 2023. 9.3 NVLink-C2C: A Coherent Off Package Chip-to-Chip Interconnect with 40Gbps/pin Single-ended Signaling. In *Proceedings of the 2023 International Solid-State Circuits Conference (ISSCC)*.
- [149] Ronald Weiss. 2012. A Technical Overview of the Oracle Exadata Database Machine and Exadata Storage Server. *Oracle White Paper* (2012).
- [150] Linus Y. Wong, Jialiang Zhang, and Jing (Jane) Li. 2023. DONGLE: Direct FPGA-orchestrated NVMe storage for HLS. In *Proceedings of the 2023 International Symposium on Field-Programmable Gate Arrays (FPGA)*.
- [151] Haojun Xia, Zhen Zheng, Yuchao Li, Donglin Zhuang, Zhongzhu Zhou, Xiafei Qiu, Yong Li, Wei Lin, and Shuaiwen Leon Song. 2023. Flash-LLM: Enabling Cost-Effective and Highly-Efficient Large Generative Model Inference with Unstructured Sparsity. In *Proceedings of the 49th International Conference on Very Large Data Bases (VLDB)*.
- [152] Shaojie Xiang, Yi-Hsiang Lai, Yuan Zhou, Hongzheng Chen, Niansong Zhang, Debjit Pal, and Zhiru Zhang. 2022. HeteroFlow: An Accelerator Programming Model with Decoupled Data Placement for Software-Defined FPGAs. In *Proceedings of the 2022 International Symposium on Field-Programmable Gate Arrays (FPGA)*.
- [153] Xilinx. [n. d.]. AMD Kintex™ UltraScale+™ FPGAs. <https://www.amd.com/en/products/adaptive-socs-and-fpgas/fpga/kintex-ultrascale-plus.html>. Accessed: 2025-12-12.
- [154] Xilinx. [n. d.]. Xilinx OpenCL Extension. https://xilinx.github.io/XRT/master/html/opencl_extension.html. Accessed: 2025-12-12.
- [155] W. Xiong, L. Ke, D. Jankov, M. Kounavis, X. Wang, E. Northup, J. Yang, B. Acun, C. Wu, P. Peter Tang, G. Edward Suh, X. Zhang, and H. S. Lee. 2022. SecNDP: Secure Near-Data Processing with Untrusted Memory. In *Proceedings of the 2022 International Symposium on High-Performance Computer Architecture (HPCA)*.
- [156] Dong Xu, Yuan Feng, Kwangsik Shin, Daewoo Kim, Hyeran Jeon, and Dong Li. 2024. Efficient Tensor Offloading for Large Deep-Learning Model Training based on Compute Express Link. In *SC'24: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*.
- [157] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A Distributed Serving System for Transformer-Based Generative Models. In *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.
- [158] Zhongkai Yu, Shengwen Liang, Tianyun Ma, Yunke Cai, Ziyuan Nan, Di Huang, Xinkai Song, Yifan Hao, Jie Zhang, Tian Zhi, Yongwei Zhao, Zidong Du, Xing Hu, Qi Guo, and Tianshi Chen. 2024. Cambricon-LLM: A Chiplet-Based Hybrid Architecture for On-Device Inference of 70B LLM. In *Proceedings of the 57th Annual International Symposium on Microarchitecture (MICRO)*.
- [159] Sungmin Yun, Kwanhee Kyung, Juhwan Cho, Jaewan Choi, Jongmin Kim, Byeongho Kim, Sukhan Lee, Kyomin Sohn, and Jung Ho Ahn. 2024. Duplex: A Device for Large Language Models with Mixture of Experts, Grouped Query Attention, and Continuous Batching. In *Proceedings of the 57th Annual International Symposium on Microarchitecture (MICRO)*.
- [160] Jie Zhang, David Donofrio, John Shalf, Mahmut T. Kandemir, and Myoungsoo Jung. 2015. NVMMU: A non-volatile memory management unit for heterogeneous GPU-SSD architectures. In *Proceedings of the 2015 International Conference on Parallel Architecture and Compilation (PACT)*.
- [161] Jian Zhang, Yujie Ren, Marie Nguyen, Changwoo Min, and Sudarsun Kannan. 2024. OmniCache: Collaborative Caching for Near-storage Accelerators. In *Proceedings of the 22nd USENIX Conference on File and Storage Technologies (FAST)*.
- [162] Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, et al. 2022. Opt: Open Pre-Trained Transformer Language Models. *arXiv preprint arXiv:2205.01068* (2022).
- [163] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang "Atlas" Wang, and Beidi Chen. 2023. H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models. In *Proceedings of the Advances in Neural Information Processing Systems 36 (NeurIPS)*.
- [164] Xuanlei Zhao, Bin Jia, Haotian Zhou, Ziming Liu, Shenggan Cheng, and Yang You. 2024. HeteGen: Heterogeneous Parallel Inference for Large Language Models on Resource-Constrained Devices. *arXiv preprint arXiv:2403.01164* (2024).
- [165] Yilong Zhao, Chien-Yu Lin, Kan Zhu, Zihao Ye, Lequn Chen, Size Zheng, Luis Ceze, Arvind Krishnamurthy, Tianqi Chen, and Baris Kasikci. 2023. Atom: Low-bit quantization for efficient and accurate llm serving. *arXiv preprint arXiv:2310.19102* (2023).
- [166] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, Hao Zhang, Joseph E Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Proceedings of the Advances in Neural Information Processing Systems 36 (NeurIPS)*.
- [167] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. *arXiv preprint arXiv:2401.09670* (2024).
- [168] Minxuan Zhou, Weihong Xu, Jaeyoung Kang, and Tajana Rosing. 2022. TransPIM: A Memory-based Acceleration via Software-Hardware Co-Design for Transformer. In *Proceedings of the 2022 International Symposium on High Performance Computer Architecture (HPCA)*.